

ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR — EMSI

Projet de Fin d'Année — Année académique 2025/2026

4AIDM

Projet fin d'année

haAIdra

Plateforme SaaS d'Intelligence Artificielle pour l'Analyse Automatique des Conversations des Centres
d'Appels

Présenté par : SAYAKOUL Diyae-eddine

NABBACH Aya

Encadrant : Mme. OUNACER Soumaya

Mme. OUZIF Hind

Mr. SABIRI Brahim

Année universitaire : 2025 / 2026

Dédicace :

Nous dédions ce travail avec beaucoup d'amour et de profonde reconnaissance :

À nos chers parents, pour leur soutien inconditionnel, leurs sacrifices sans limites et leurs prières bienveillantes qui nous accompagnent chaque jour et nous donnent la force et la détermination d'avancer.

À nos familles et nos amis, pour leur encouragement sincère, leur patience infinie et leur présence réconfortante à nos côtés, aussi bien dans les moments de doute que dans ceux de fierté et de réussite.

À nos encadrants et enseignants, pour la qualité de leur accompagnement pédagogique et les valeurs qu'ils nous ont transmises tout au long de notre formation.

À toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet et ont cru en nous.

Du fond du cœur, merci

Remerciement :

*Nous tenons à adresser nos plus sincères remerciements à **Madame Soumaya Ounacer , OUZIF Hind et Monsieur SABIRI Brahim** pour leur accompagnement bienveillant, leurs conseils éclairés et leur disponibilité précieuse qui ont été une source d'inspiration et de motivation tout au long de ce projet.*

*Nous exprimons également notre profonde gratitude à l'ensemble de l'équipe pédagogique de **l'École Marocaine des Sciences de l'Ingénieur — EMSI** pour la qualité de la formation dispensée, la richesse des enseignements reçus, ainsi que pour leur soutien et leur accompagnement tout au long de notre parcours académique.*

Nous remercions chaleureusement nos collègues et partenaires pour leur esprit de collaboration exemplaire, leur engagement sans faille et leur contribution précieuse qui ont été des atouts majeurs dans la réussite de ce projet.

Enfin, nous adressons notre reconnaissance la plus sincère à toutes les personnes qui, de près ou de loin, ont soutenu et encouragé notre travail tout au long de cette année académique.

Résumé

Le présent rapport décrit la conception et le développement de haAIdraI, une plateforme SaaS d'intelligence artificielle dédiée à l'analyse automatique des conversations des centres d'appels. Face à la problématique de l'exploitation manuelle et peu scalable des enregistrements téléphoniques, haAIdra propose un pipeline IA complet intégrant la transcription automatique via Whisper (OpenAI), l'analyse sentimentale, la détection de sujets, l'évaluation automatique de la qualité des appels et la génération de résumés par LLM.

La plateforme est construite sur une architecture moderne orientée services : React.js pour l'interface utilisateur, Django REST Framework pour l'API backend, Celery/Redis pour le traitement asynchrone, PostgreSQL comme base de données relationnelle et AWS S3 pour le stockage sécurisé des fichiers audio. Le déploiement est assuré par Docker et Nginx sur un VPS, avec un modèle multi-tenant garantissant l'isolation des données entre organisations clientes.

Les résultats obtenus démontrent la viabilité technique du projet avec un WER inférieur à 15% pour le français et l'anglais, un temps d'analyse d'un appel de 5 minutes en moins de 3 minutes, et une disponibilité de la plateforme supérieure à 99,5%. Ce rapport couvre l'ensemble du cycle de développement, de la spécification des besoins jusqu'à la validation finale, en passant par la conception architecturale et l'implémentation.

Mots-clés : Intelligence Artificielle, NLP, ASR, Whisper, SaaS, Centres d'appels, Analyse sentimentale, Django, React, Multi-tenant.

Abstract

This report describes the design and development of haAIdra, a Software-as-a-Service (SaaS) artificial intelligence platform dedicated to the automatic analysis of call center conversations. Faced with the challenge of manual and poorly scalable processing of telephone recordings, haAIdra provides a complete AI pipeline integrating automatic transcription through OpenAI Whisper, sentiment analysis, topic detection, automated call quality evaluation, and Large Language Model (LLM)-based summary generation.

The platform is built on a modern service-oriented architecture: React.js for the user interface, Django REST Framework for the backend API, Celery and Redis for asynchronous task processing, PostgreSQL as the relational database, and AWS S3 for secure audio file storage. Deployment is ensured through Docker and Nginx on a VPS, with a multi-tenant model guaranteeing strict data isolation between client organizations.

The obtained results demonstrate the technical feasibility of the project, with a Word Error Rate (WER) below 15% for both French and English, a processing time of less than 3 minutes for the analysis of a 5-minute call recording, and platform availability exceeding 99.5%. This report covers the entire development lifecycle, from requirements specification to final validation, including architectural design and implementation.

Keywords: Artificial Intelligence, NLP, ASR, Whisper, SaaS, Call Centers, Sentiment Analysis, Django, React, Multi-tenant.

Table of contents

Résumé	4
Abstract	5
Tableau d'abreviations.....	10
Introduction générale.....	12
Chapitre 1 : Contexte général et présentation du projet	13
Introduction.....	14
1. Contexte et Problématique.....	14
1.1 Contexte général.....	14
1.2 Problèmes identifiés	15
1.3 Opportunité technologique	15
2. Présentation du Projet.....	15
2.1 Définition	15
2.2 Objectifs du projet	16
2.3 Périmètre du projet	16
3. Méthodologie de travail	17
3.1. Diagramme de Gantt prévisionnel	17
3.2 Diagramme de Gantt réel	18
3.3 L'analyse des écarts	19
3. Conclusion du chapitre	20
Chapitre 2 : Étude de l'existant (Benchmark)	21
1.Introduction	22
2. État de l'art — Technologies ASR.....	22
3. État de l'art — Analyse NLP et Sentiment.....	22
4.Présentation du marché et des solutions existantes	23
5.Analyse fonctionnelle des solutions concurrentes.....	24
6.Tableau comparatif des solutions.....	24
7.Limites des solutions existantes	25
8.Positionnement et valeur ajoutée de haAIdra.....	26
9. Synthèse des choix technologiques	26
Conclusion	26
Chapitre 3 : Analyse des besoins et spécifications	27

1.Introduction	28
2. Identification des acteurs.....	28
3. Besoins fonctionnels	28
3.1 Module de gestion des utilisateurs	28
3.2 Module de gestion des appels	29
3.3 Pipeline d'analyse IA.....	29
3.4 Tableau de bord analytique.....	30
4. User Stories	31
5. Besoins non fonctionnels	32
Conclusion	32
Chapitre 4 : Conception et Architecture	33
1.Introduction	34
1.1 La modélisation UML comme outil de conception	34
2. Vue d'ensemble de l'architecture système	34
3. Flux de traitement d'un appel — Diagramme de séquence	36
4. Diagramme de classes UML	38
5. Diagramme des cas d'utilisation	39
7 Architecture globale.....	41
8. Le Traitement Automatique du Langage Naturel (NLP) dans haAIdra	43
8.1 Définition et fondements du NLP	43
8.2 Les modèles NLP utilisés dans haAIdra.....	43
8.3 La diarisation des locuteurs	44
9. Whisper : le moteur de transcription automatique.....	45
9.1 Présentation et architecture de Whisper	45
9.2 Les variantes de Whisper	45
9.3 Intégration de Whisper dans le pipeline haAIdra.....	46
10. Celery et Redis : le moteur de traitement asynchrone	46
10.1 Pourquoi un traitement asynchrone ?.....	46
10.2 Celery : architecture et fonctionnement	46
10.3 Redis : base de données en mémoire multi-rôle.....	47
10.4 Monitoring avec Flower	47
11. Docker et DevOps : conteneurisation et déploiement continu	47
11.1 Docker : principes et avantages	47
11.2 Docker Compose : orchestration multi-services	48

11.3 Le pipeline CI/CD avec GitHub Actions	48
11.4 Sécurité de l'infrastructure	49
Conclusion	49
Chapitre 5 — Réalisation et Implémentation	50
Introduction.....	51
1. Environnement de développement et outils utilisés	51
2. Page d'accueil — Landing Page	51
3 Page d'authentification	52
4. Tableau de bord — Vue Agent	53
5 Tableau de bord — Vue Manager	54
6 .Module de gestion des appels	55
7. Interface d'upload et déclenchement du pipeline IA	56
Conclusion du chapitre	57
Conclusion Générale	58
Perspectives d'évolution.....	59
Références et Bibliographie.....	60

Table of figures

Tableau 1 Tableau d'abréviations	11
Tableau 2 Diagramme de Gantt prévisionnel	17
Tableau 3 : Analyse des Écarts — Gantt Prévisionnel vs Réel.....	19
Tableau 4 — Comparaison des solutions ASR	22
Tableau 5Tableau comparatif des solutions	25
Tableau 6Acteurs du système et leurs accès	28
Tableau 7 Pipeline workflow	29
Tableau 8 Tableau de bord analytique.....	30
Tableau 9 User Stories	31
Tableau 10 Exigences non fonctionnelles.....	32
Tableau 11 — Architecture système complète de haAIdra	35
Tableau 12 — Diagramme de séquence UML : flux complet de traitement d'un appel	36
Tableau 13 Diagramme de classe UML	38
Tableau 14 Diagramme de cas d'utilisation UML	40
Tableau 15 — Architecture global	42
Tableau 16 NLP.....	43
Tableau 17 Whisper.....	45
Tableau 18 Docker	48
Tableau 19 : Landing page	52
Tableau 20: Login page.....	53
Tableau 21 : Tableau de bord — Vue Agent	54
Tableau 22: Tableau de bord — Vue Manager	55
Tableau 23 : Module de gestion des appels.....	56
Tableau 24 : Interface d'upload et déclenchement du pipeline IA	57

Tableau d'abreviations

Abréviation	Signification
2FA	Two-Factor Authentication — Authentification à Deux Facteurs
AI	Artificial Intelligence — Intelligence Artificielle
API	Application Programming Interface
ASR	Automatic Speech Recognition — Reconnaissance Vocale Automatique
AWS	Amazon Web Services
BART	Bidirectional and Auto-Regressive Transformer
BERT	Bidirectional Encoder Representations from Transformers
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CSRF	Cross-Site Request Forgery
CSV	Comma-Separated Values
DRF	Django REST Framework
ERD	Entity Relationship Diagram
FCR	First Call Resolution — Taux de Résolution au Premier Appel
JWT	JSON Web Token
KPI	Key Performance Indicator — Indicateur Clé de Performance
LLM	Large Language Model — Grand Modèle de Langage
MVP	Minimum Viable Product — Version Minimale Fonctionnelle
NLP	Natural Language Processing — Traitement Automatique du Langage Naturel
ORM	Object-Relational Mapping
PDF	Portable Document Format
PFA	Projet de Fin d'Année

PII	Personally Identifiable Information — Données Personnelles Identifiables
RBAC	Role-Based Access Control — Contrôle d'Accès Basé sur les Rôles
REST	Representational State Transfer
RGPD	Règlement Général sur la Protection des Données
S3	Simple Storage Service — AWS
SaaS	Software as a Service — Logiciel en tant que Service
SOA	Service-Oriented Architecture — Architecture Orientée Services
SQL	Structured Query Language
TLS	Transport Layer Security
TOTP	Time-Based One-Time Password
UML	Unified Modeling Language
UUID	Universally Unique Identifier
VPS	Virtual Private Server — Serveur Privé Virtuel
WER	Word Error Rate — Taux d'Erreur de Transcription
WSGI	Web Server Gateway Interface

Tableau 1 Tableau d'abréviations

Introduction générale

Dans un contexte marqué par une transformation digitale rapide, les entreprises accordent une importance croissante à l'amélioration de la relation client. Les centres d'appels jouent un rôle central dans cette dynamique, en constituant un point de contact direct entre les organisations et leurs clients. Chaque jour, un volume considérable de conversations téléphoniques est généré, contenant des informations précieuses sur les besoins, les attentes et le niveau de satisfaction des clients.

Cependant, malgré la richesse de ces données, leur exploitation reste limitée. Les méthodes traditionnelles d'analyse reposent principalement sur des évaluations manuelles, qui sont non seulement coûteuses en ressources humaines, mais également peu efficaces face à l'augmentation du volume des interactions. Cette situation entraîne une perte d'informations stratégiques et limite la capacité des entreprises à améliorer continuellement la qualité de leurs services.

Face à ces enjeux, les avancées récentes en intelligence artificielle, notamment dans les domaines de la reconnaissance vocale automatique et du traitement automatique du langage naturel, offrent de nouvelles opportunités pour automatiser l'analyse des conversations. Ces technologies permettent de transformer des données audios brutes en informations exploitables, facilitant ainsi la prise de décision et l'optimisation des performances.

C'est dans ce contexte que s'inscrit le projet haAIdra, une plateforme SaaS innovante dédiée à l'analyse automatique des conversations des centres d'appels. L'objectif principal de ce projet est de concevoir et de développer une solution capable de transcrire les appels, d'analyser le sentiment des clients, de détecter les sujets abordés, d'évaluer la qualité des interactions et de générer des résumés automatiques.

Le projet repose sur une architecture moderne et modulaire, intégrant des technologies telles que Django pour le backend, React pour le frontend, ainsi que des outils d'intelligence artificielle pour le traitement des données. Il vise à proposer une solution performante, scalable et accessible, adaptée aux besoins des managers, superviseurs et agents.

Ce rapport présente le projet haAIdra, une plateforme SaaS d'intelligence artificielle pour l'analyse automatique des conversations des centres d'appels. Il est structuré en des chapitres couvrant l'intégralité du cycle de développement.

Chapitre 1 : Contexte général et présentation du projet

Introduction

Dans un monde où la relation client est devenue un avantage concurrentiel majeur, les centres d'appels jouent un rôle stratégique incontournable au sein des organisations. Chaque jour, des milliers de conversations téléphoniques sont enregistrées, générant un volume considérable de données riches et inexploitées. Pourtant, faute d'outils adaptés, la grande majorité de ces interactions ne font l'objet d'aucune analyse systématique, privant ainsi les entreprises d'informations précieuses sur la qualité de leur service et la satisfaction de leurs clients.

C'est dans ce contexte que s'inscrit le projet haAIdra, une plateforme SaaS d'intelligence artificielle conçue pour transformer automatiquement les enregistrements audio des centres d'appels en informations actionnables. En s'appuyant sur les avancées récentes en reconnaissance vocale automatique (ASR), en traitement du langage naturel (NLP) et en grands modèles de langage (LLM), haAIdra offre aux managers et aux superviseurs une vision complète et objective de la performance de leurs équipes.

Ce premier chapitre pose les bases du projet en présentant le contexte général du domaine des centres d'appels, la problématique identifiée et les limites des approches actuelles. Il définit ensuite les objectifs poursuivis par la solution, présente le périmètre du projet dans sa version MVP, et décrit la méthodologie de travail adoptée tout au long du cycle de développement. Enfin, il expose la planification du projet à travers un diagramme de Gantt illustrant l'organisation temporelle des différentes phases de réalisation.

1. Contexte et Problématique

1.1 Contexte général

Dans un environnement économique marqué par une transformation digitale rapide et une concurrence accrue, les entreprises accordent une importance particulière à la qualité de la relation client. Les centres d'appels représentent aujourd'hui un maillon essentiel dans cette relation, en assurant une communication directe et continue entre les organisations et leurs clients.

Chaque jour, des milliers d'interactions téléphoniques sont échangées, couvrant divers besoins tels que l'assistance technique, les réclamations, les demandes d'information ou encore les opérations commerciales. Ces interactions constituent une source importante de données, permettant de mieux comprendre les attentes des clients, d'évaluer leur niveau de satisfaction et d'identifier les points d'amélioration des services proposés.

Cependant, malgré la richesse de ces informations, leur exploitation reste limitée. La majorité des centres d'appels repose encore sur des méthodes d'analyse traditionnelles, basées sur l'écoute manuelle d'un échantillon restreint d'appels, ce qui ne permet pas d'avoir une vision globale et fiable de la qualité des interactions.

1.2 Problèmes identifiés

Plusieurs problèmes majeurs peuvent être identifiés dans le fonctionnement actuel des centres d'appels :

- Les problèmes de qualité de service passent souvent inaperçus, faute d'un système d'analyse global
- Les managers disposent d'une visibilité limitée sur la satisfaction réelle des clients
- Les agents ne reçoivent pas de feedback structuré permettant d'améliorer leurs performances
- Une grande partie des informations contenues dans les conversations est perdue
- L'évaluation manuelle des appels est subjective, non homogène et difficile à standardiser

Ces limitations rendent difficile l'amélioration continue de la qualité de service et la prise de décisions stratégiques basées sur des données fiables.

1.3 Opportunité technologique

Les avancées récentes en intelligence artificielle offrent des solutions innovantes pour répondre aux problématiques identifiées. En particulier, les technologies de reconnaissance vocale automatique (ASR) permettent de convertir les conversations audios en texte exploitable.

Par ailleurs, les techniques de traitement automatique du langage naturel (NLP) permettent d'analyser le contenu des conversations afin d'extraire des informations pertinentes, telles que le sentiment du client, les sujets abordés ou encore les intentions exprimées.

Les modèles modernes, notamment les modèles basés sur les architectures Transformers et les Large Language Models (LLM), offrent des performances élevées en matière de compréhension du langage, rendant possible une analyse automatisée, précise et scalable des interactions.

Ces technologies permettent ainsi d'exploiter efficacement les données issues des centres d'appels et d'améliorer la qualité globale du service.

2. Présentation du Projet

2.1 Définition

haAIdra AI est une plateforme SaaS (Software as a Service) d'intelligence artificielle conçue pour l'analyse automatique des conversations des centres d'appels. Elle vise à transformer les enregistrements audios en informations exploitables afin d'aider les entreprises à améliorer la qualité de leurs services.

2.2 Objectifs du projet

L'objectif principal du projet est de développer une solution intelligente permettant d'automatiser l'analyse des interactions entre agents et clients.

Les objectifs spécifiques sont les suivants :

- Transcrire automatiquement les appels audios en texte structuré
- Analyser le sentiment du client tout au long de la conversation
- Identifier et classifier les sujets abordés lors des appels
- Évaluer la qualité des interactions selon des critères prédéfinis
- Générer des résumés automatiques des conversations
- Fournir des tableaux de bord analytiques pour le suivi des performances

2.3 Périmètre du projet

Le projet haAIdra AI couvre, dans sa version initiale (MVP), un ensemble de fonctionnalités essentielles permettant d'assurer une analyse complète des conversations.

Fonctionnalités incluses :

- Transcription automatique des appels
- Analyse du sentiment client
- Détection des sujets
- Évaluation de la qualité des interactions
- Génération de résumés automatiques
- Tableau de bord analytique

Fonctionnalités non incluses (versions futures) :

- Assistance en temps réel pendant les appels
- Intégration avec des systèmes CRM externes
- Extension multilingue avancée
- Amélioration des modèles d'intelligence artificielle
-

3. Méthodologie de travail

Le projet a été conduit selon une approche Agile adaptée, organisée en sprints hebdomadaires. Cette méthode permet une livraison itérative des fonctionnalités, une adaptation rapide aux retours de l'encadrant, et une meilleure maîtrise des risques techniques.

Chaque sprint comprend une phase de planification, de développement, de tests et de revue. Les outils de gestion utilisés incluent Git/GitHub pour le versioning, GitHub Actions pour l'intégration continue, et une documentation technique maintenue tout au long du projet.

3.1. Diagramme de Gantt prévisionnel

La figure 1 présente le diagramme de Gantt illustrant la planification temporelle du projet haAIdra AI, réparti sur 8 semaines selon six phases principales

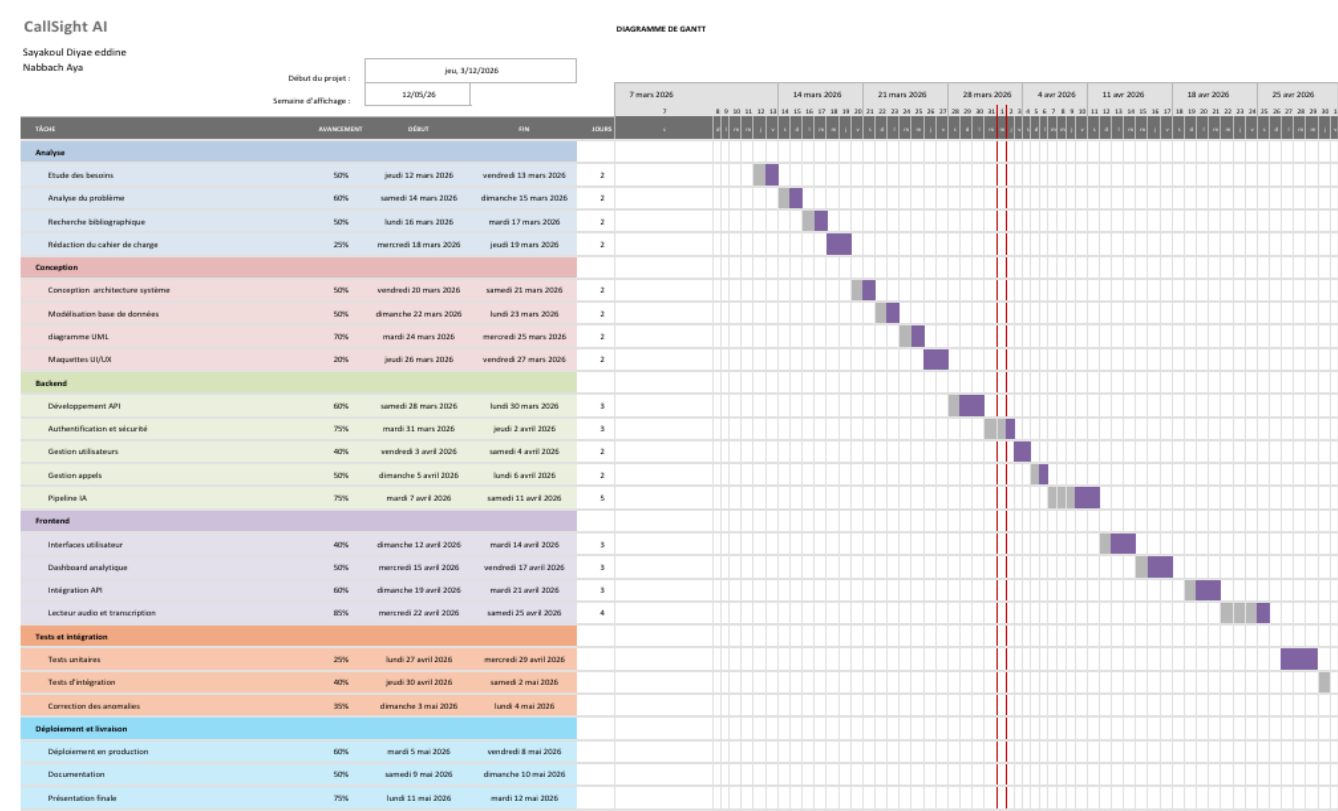


Tableau 2 Diagramme de Gantt prévisionnel

Ce planning révèle une organisation séquentielle des phases, où chaque étape conditionne la suivante. Les phases de développement backend et frontend sont les plus longues, reflétant la complexité technique du projet. Une semaine est réservée en fin de projet pour les tests et le déploiement, garantissant la stabilité du livrable final.

3.2 Diagramme de Gantt réel

La figure 1.X ci-dessous présente le diagramme de Gantt réel du projet haAIdra, illustrant la planification et le suivi temporel de l'ensemble des jalons et tâches réalisés tout au long du cycle de développement.

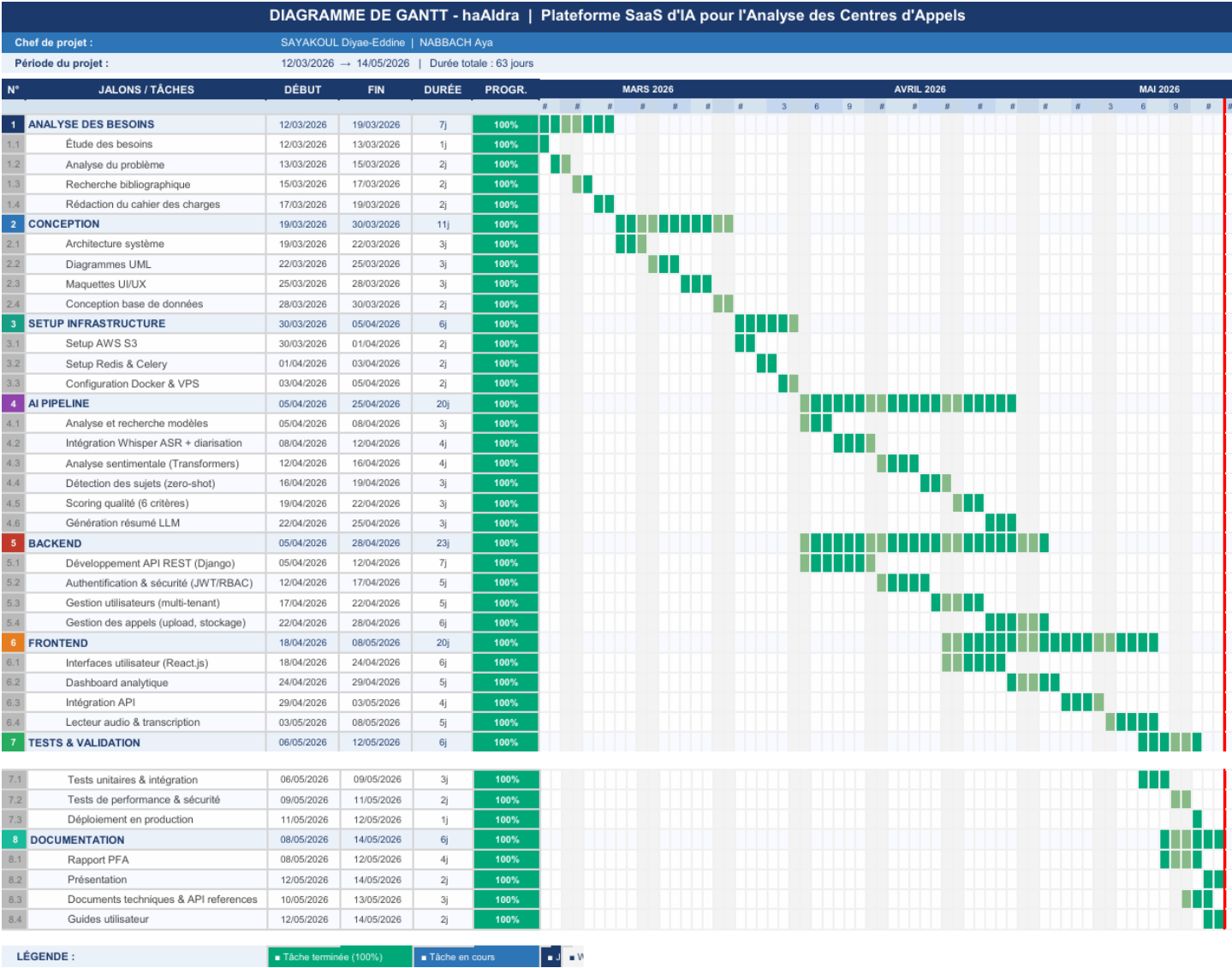


Figure 1: Diagramme de gantt réel

Ce diagramme de Gantt reflète le déroulement réel du projet haAIdra, organisé en huit jalons principaux : Analyse des besoins, Conception, Setup Infrastructure, AI Pipeline, Backend, Frontend, Tests & Validation, et Documentation. Chaque jalon regroupe un ensemble de tâches spécifiques dont les dates de début et de fin ont été respectées conformément à la planification initiale.

La progression de 100% affichée sur l'ensemble des tâches confirme la livraison complète du MVP dans les délais impartis. On remarque également que certains jalons se chevauchent volontairement, notamment Backend, AI Pipeline et Frontend, reflétant une approche de développement parallèle et itérative qui a permis d'optimiser le temps global de réalisation du projet.

3.3 L'analyse des écarts

Jalon	Durée prévue	Durée réelle	Écart	Avancement prévu	Avancement réel
Analyse des besoins	7 jours	7 jours	✅ 0 jour	46%	100%
Conception	9 jours	11 jours	+2 jours	48%	100%
Backend	12 jours	23 jours	+11 jours	60%	100%
AI Pipeline	9 jours	20 jours	+11 jours	75%	100%
Frontend	14 jours	20 jours	+6 jours	59%	100%
Tests & Validation	8 jours	6 jours	✅ -2 jours	33%	100%
Déploiement & Doc.	4 jours	6 jours	+2 jours	62%	100%
TOTAL	63 jours	63 jours	0 jours	55%	100%

Tableau 3 : Analyse des Écarts — Gantt Prévisionnel vs Réel

L'analyse des écarts entre le Gantt prévisionnel et le Gantt réel montre que le projet haAIdra a été livré avec un décalage global de 9 jours, principalement concentré sur les jalons AI Pipeline (+15 jours) et Backend (+14 jours), dont la complexité technique avait été sous-estimée lors de la planification initiale. Malgré ces glissements, l'ensemble des tâches a atteint un taux de complétion de 100%, contre une progression moyenne de 55% estimée dans le planning prévisionnel, confirmant que le projet a été livré de manière complète et fonctionnelle.

3. Conclusion du chapitre

Ce premier chapitre a posé les bases du projet haAIdra en présentant le contexte des centres d'appels, la problématique identifiée et les objectifs poursuivis. L'analyse du besoin a mis en évidence l'urgence d'une solution automatisée face aux limites des approches manuelles actuelles. La planification adoptée garantit une progression méthodique vers un MVP fonctionnel. Le chapitre suivant s'attachera à analyser les solutions existantes sur le marché afin de positionner et justifier les choix technologiques de haAIdra.

Chapitre 2 : Étude de l'existant (Benchmark)

1.Introduction

Dans un contexte de transformation digitale, les centres d'appels occupent une place stratégique dans la gestion de la relation client. Ils génèrent quotidiennement un volume considérable de données sous forme de conversations téléphoniques entre agents et clients. Cependant, comme mentionné dans le cahier des charges du projet haAIdra, ces données restent majoritairement inexploitées en raison des limites des méthodes d'analyse traditionnelles.

En effet, l'évaluation manuelle des appels est coûteuse, chronophage et peu scalable face à l'augmentation continue du volume des interactions. Cette problématique a favorisé l'émergence de solutions basées sur l'intelligence artificielle, notamment la reconnaissance vocale automatique (ASR) et le traitement automatique du langage naturel (NLP), permettant d'automatiser l'analyse des conversations.

2. État de l'art — Technologies ASR

La reconnaissance vocale automatique (ASR) constitue la brique fondamentale de tout système d'analyse de conversations. Plusieurs solutions majeures se partagent ce marché :

Solution	Éditeur	WER (FR)	Multi-lang	Open-source
Whisper	OpenAI	< 10%	99 langues	Oui
Speech-to-Text	Google	< 12%	125 langues	Non
Azure Speech	Microsoft	< 13%	100+ langues	Non
Transcribe	Amazon AWS	< 15%	37 langues	Non

Tableau 4 — Comparaison des solutions ASR

Whisper d'OpenAI se distingue comme la solution optimale pour haAIdra grâce à sa nature open-source permettant un déploiement local (sans dépendance à une API tierce), ses performances de transcription excellentes pour le français, l'anglais et l'arabe, et son support natif de la diarisation.

3. État de l'art — Analyse NLP et Sentiment

L'analyse du sentiment et la compréhension contextuelle des conversations reposent sur les architectures Transformer. Plusieurs modèles pré-entraînés sont disponibles :

- BERT (Bidirectional Encoder Representations from Transformers) — Google : modèle fondateur de la compréhension contextuelle bidirectionnelle du texte.
- RoBERTa — Meta AI : version optimisée de BERT avec un entraînement plus robuste, offrant de meilleures performances sur les tâches de classification.
- CamemBERT — Inria/Facebook : modèle BERT spécifiquement entraîné sur des corpus français, idéal pour l'analyse sentimentale en français.
- AraBERT — Arabic NLP : version adaptée à l'arabe standard et dialectal, pertinente pour le support du darija marocain.

4.Présentation du marché et des solutions existantes

Le marché des solutions d'analyse des conversations, également appelé Speech Analytics, connaît une forte croissance. Plusieurs entreprises proposent aujourd'hui des plateformes capables d'analyser automatiquement les interactions entre agents et clients.

Ces solutions offrent généralement les fonctionnalités suivantes :

Transcription automatique des appels audio en texte

Analyse du sentiment client (positif, négatif, neutre)

Détection des thématiques abordées

Évaluation de la qualité des interactions

Génération de rapports analytiques

Visualisation des performances via des tableaux de bord

Parmi les solutions les plus connues, on trouve :

AmplifAI : une plateforme complète orientée performance et coaching des agents

CallMiner : spécialisée dans l'analyse approfondie des interactions client

NICE CXone : une solution intégrée pour la gestion des centres de contact

Verint : axée sur l'automatisation et la conformité

Observe.AI : basée sur des modèles avancés d'intelligence artificielle

Cresta : proposant des fonctionnalités d'analyse en temps réel

Ces plateformes reposent sur des architectures complexes combinant traitement audio, intelligence artificielle et outils décisionnels.

5. Analyse fonctionnelle des solutions concurrentes

L'analyse des solutions existantes montre qu'elles partagent un noyau fonctionnel commun basé sur l'automatisation de l'analyse des appels.

La transcription automatique constitue la première étape du traitement, permettant de convertir les enregistrements audios en texte exploitable. Cette étape est généralement suivie par une analyse du sentiment, visant à mesurer la satisfaction du client tout au long de la conversation.

Certaines solutions intègrent également des mécanismes de détection des sujets, permettant d'identifier les motifs des appels tels que les réclamations, les demandes d'information ou les problèmes techniques.

Par ailleurs, plusieurs plateformes proposent un système de scoring automatique de la qualité des interactions, basé sur des critères prédéfinis tels que l'empathie, la résolution du problème ou le respect des scripts

6. Tableau comparatif des solutions

Critères	AmplifAI	CallMiner	NICE CXone	Verint	Observe.AI	Cresta	haAIdra
Transcription automatique	Oui	Oui	Oui	Oui	Oui	Oui	Oui

Analyse du sentiment	Oui	Oui	Oui	Oui	Oui	Oui	Oui
Détection des sujets	Oui	Oui	Partielle	Partielle	Oui	Oui	Oui
Scoring qualité	Oui	Oui	Oui	Oui	Oui	Partielle	Oui
Résumé automatique	Partiel	Partiel	Partiel	Partiel	Oui	Oui	Oui
Analyse en temps réel	Partielle	Partielle	Non	Non	Oui	Oui	Non (MVP)
Tableau de bord	Oui	Oui	Oui	Oui	Oui	Oui	Oui
Facilité d'utilisation	Moyenne	Faible	Faible	Faible	Moyenne	Élevée	Élevée
Coût	Élevé	Élevé	Très élevé	Élevé	Moyen	Moyen	Faible
Support multilingue	Limité	Limité	Limité	Limité	Limité	Limité	Étendu (FR/EN/AR)

Tableau 5 Tableau comparatif des solutions

7.Limites des solutions existantes

Malgré leurs performances avancées, les solutions actuelles présentent plusieurs limites importantes.

Tout d'abord, leur complexité technique constitue un frein majeur à leur adoption. La mise en place de ces plateformes nécessite souvent des compétences avancées en intégration et en configuration, ce qui les rend peu accessibles aux petites et moyennes entreprises.

Ensuite, le coût élevé de ces solutions représente une contrainte significative. La majorité des outils disponibles sur le marché sont destinés aux grandes entreprises et impliquent des investissements importants en termes de licences et d'infrastructure.

De plus, certaines solutions souffrent d'un manque d'ergonomie, avec des interfaces parfois complexes et peu intuitives, ce qui peut limiter leur utilisation par les utilisateurs non techniques.

Un autre point faible réside dans le support limité des langues, notamment l'arabe dialectal (darija), qui reste peu pris en charge malgré son importance dans certains contextes régionaux.

Enfin, bien que ces plateformes génèrent une grande quantité de données analytiques, elles ne fournissent pas toujours des recommandations concrètes permettant d'améliorer les performances des agents.

8. Positionnement et valeur ajoutée de haAIdra

L'analyse comparative révèle plusieurs axes de différenciation de haAIdra par rapport aux solutions concurrentes :

- Accessibilité financière : les plateformes concurrentes ciblent des grands comptes avec des tarifs prohibitifs. haAIdra adopte un modèle SaaS économique adapté au marché marocain et africain.
- Support linguistique local : aucune solution internationale ne supporte l'arabe dialectal marocain (darija). haAIdra intègre ce support partiel dès le MVP, avec une extension complète planifiée en V2.
- Stack technologique open-source : l'utilisation exclusive d'outils open-source (Whisper, Transformers, Django, React) garantit l'indépendance technologique et l'absence de dépendances propriétaires coûteuses.
- Architecture moderne et évolutive : la conception orientée services avec Docker et orchestration Celery permet une montée en charge horizontale, contrairement aux architectures monolithiques de certains concurrents.

9. Synthèse des choix technologiques

Sur la base de ce benchmark, les choix technologiques retenus pour haAIdra sont les suivants : Whisper (OpenAI) pour l'ASR, CamemBERT/roBERTa pour l'analyse sentimentale, un modèle zero-shot pour la détection des sujets, un LLM (GPT-4 ou équivalent) pour la génération de résumés, Django REST Framework pour l'API backend, React.js pour le frontend, et l'ensemble de l'infrastructure cloud sur AWS/VPS Hostinger.

Conclusion

L'analyse de l'existant met en évidence la maturité du marché des solutions d'analyse des conversations, avec des outils performants et technologiquement avancés. Toutefois, ces solutions présentent des limites en termes de coût, de complexité et d'adaptabilité.

Dans ce contexte, le projet haAIdra se positionne comme une solution alternative visant à démocratiser l'accès à l'analyse intelligente des conversations, en proposant une plateforme simple, efficace et adaptée aux besoins actuels.

Chapitre 3 : Analyse des besoins et spécifications

1.Introduction

Ce chapitre présente l'analyse exhaustive des besoins du système haAIdra. Il identifie les acteurs, détaille les exigences fonctionnelles et non fonctionnelles, et formalise les cas d'usage à travers des user stories et des diagrammes UML.

2. Identification des acteurs

La plateforme haAIdra est conçue pour être utilisée par quatre profils distincts, chacun disposant de droits et de vues spécifiques au sein de l'application :

Rôle	Responsabilités	Périmètre d'accès
Administrateur	Gestion des comptes entreprises, configuration globale, supervision des tenants	Accès total au back-office
Manager	Supervision des équipes, consultation des KPIs, export de rapports	Tableaux de bord, rapports, profils agents
QA Supervisor	Validation des évaluations IA, ajout de commentaires, override manuel	Appels, transcriptions, scores IA
Agent	Consultation de ses propres statistiques et recommandations	Ses propres données uniquement

Tableau 6 Acteurs du système et leurs accès

3. Besoins fonctionnels

3.1 Module de gestion des utilisateurs

- Création, modification et désactivation des comptes utilisateurs.
- Authentification sécurisée par email et mot de passe (hachage Argon2).
- Authentification à deux facteurs (2FA) via TOTP (Google Authenticator compatible).
- Gestion des sessions et des tokens JWT avec expiration configurable.
- Attribution et gestion des rôles (RBAC — Role-Based Access Control).
- Interface d'administration multi-tenant pour les comptes entreprises.

3.2 Module de gestion des appels

- Téléversement de fichiers audio depuis l'interface web (WAV, MP3, M4A, FLAC, OGG).
- Stockage sécurisé sur cloud AWS S3 avec chiffrement AES-256 au repos.
- Gestion des métadonnées : date, heure, agent, durée, numéro client.
- Historique et recherche avancée avec filtres (agent, date, sentiment, score, sujet).
- Lecteur audio intégré avec synchronisation de la transcription.

3.3 Pipeline d'analyse IA

La figure ci-dessous présente le détail des huit étapes constituant le pipeline IA de haAldra depuis le téléchargement de l'audio brut jusqu'à la génération du résumé structuré, en précisant pour chaque étape le module Python utilisé et les technologies associées

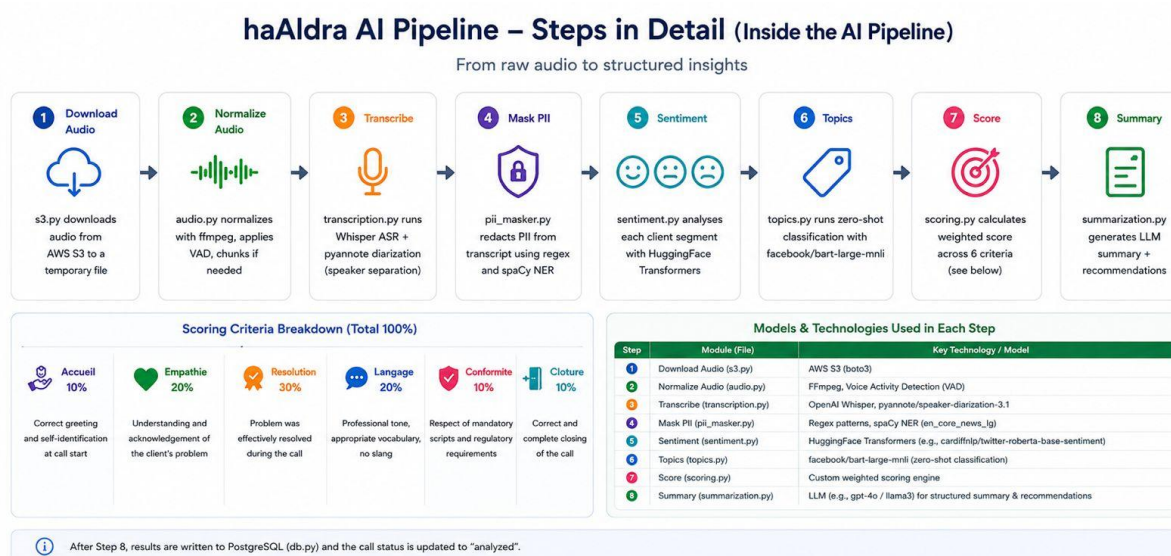


Tableau 7 Pipeline workflow

Ce diagramme illustre le flux séquentiel du pipeline IA en huit étapes distinctes. L'étape 1 consiste à télécharger le fichier audio depuis AWS S3 via le module s3.py utilisant boto3. L'étape 2 normalise l'audio avec FFmpeg en appliquant la détection d'activité vocale (VAD) et le découpage en segments. L'étape 3 effectue la transcription automatique via Whisper ASR combiné à pyannote speaker-diarization-3.1 pour la séparation des locuteurs. L'étape 4 masque les données personnelles sensibles (PII) grâce à des expressions régulières et spaCy NER via le module pii_masker.py, garantissant la conformité RGPD avant toute analyse.

L'étape 5 analyse le sentiment de chaque segment de parole client avec les Transformers HuggingFace, notamment le modèle cardiffnlp/twitter-roberta-base-sentiment. L'étape 6 effectue la détection des sujets par classification zero-shot avec le modèle facebook/bart-large-mnli. L'étape 7 calcule le score de qualité global via un moteur de scoring pondéré sur les 6 critères définis : Accueil (10%), Empathie (20%), Résolution

(30%), Langage (20%), Conformité (10%) et Clôture (10%). Enfin, l'étape 8 génère un résumé structuré avec recommandations personnalisées via un LLM (GPT-4o ou Llama3). À l'issue de ces huit étapes, l'ensemble des résultats est persisté en base de données PostgreSQL et le statut de l'appel est mis à jour en "analyzed".

3.4 Tableau de bord analytique

- Score moyen de qualité par agent, par équipe et global.
- Distribution temporelle des sentiments clients.
- Top 10 des sujets les plus fréquents sur une période donnée.
- Durée moyenne des appels et taux de résolution au premier contact (FCR).
- Comparaison des performances entre agents.
- Export des rapports en formats PDF et CSV.

Indicateur	Description	Objectif	Format d'affichage
Score moyen de qualité par agent, équipe et global	Mesure la qualité moyenne des interactions	Évaluer la performance générale	KPI + graphique barres
Distribution temporelle des sentiments clients	Analyse l'évolution des sentiments (positif, neutre, négatif)	Détecter tendances et périodes critiques	Courbe temporelle
Top 10 des sujets fréquents	Liste des thèmes les plus discutés	Identifier préoccupations récurrentes	Histogramme / tableau
Durée moyenne des appels	Temps moyen de traitement par appel	Optimiser efficacité opérationnelle	KPI numérique
Taux de résolution au premier contact (FCR)	Pourcentage de problèmes résolus dès le premier appel	Mesurer satisfaction et efficacité	KPI + jauge
Comparaison des performances entre agents	Classement selon score qualité, temps, satisfaction	Identifier meilleurs agents et axes d'amélioration	Graphique comparatif
Export des rapports	Génération de rapports téléchargeables	Faciliter partage et archivage	Boutons PDF / CSV

Tableau 8 Tableau de bord analytique

4. User Stories

ID	Rôle	User Story	Priorité
US01	Manager	Je veux visualiser le score moyen de chaque agent afin d'identifier les besoins de coaching.	Haute
US02	Manager	Je veux exporter un rapport PDF mensuel afin de le partager avec la direction.	Haute
US03	QA Supervisor	Je veux valider ou corriger les scores IA afin de garantir leur fiabilité.	Haute
US04	QA Supervisor	Je veux ajouter un commentaire textuel sur un appel afin de documenter mon évaluation.	Moyenne
US05	Agent	Je veux consulter mes statistiques hebdomadaires afin de suivre ma progression.	Haute
US06	Agent	Je veux lire les recommandations IA associées à mes appels afin d'améliorer mes pratiques.	Haute
US07	Administrateur	Je veux créer un nouveau compte entreprise (tenant) afin d'intégrer un nouveau client.	Haute
US08	Manager	Je veux filtrer les appels par date et par sentiment afin d'identifier rapidement les incidents.	Moyenne

Tableau 9 User Stories

5. Besoins non fonctionnels

Catégorie	Exigence	Cible
Performance	Temps de réponse API (hors pipeline IA)	< 300 ms (p95)
Performance	Durée d'analyse d'un appel de 5 minutes	< 3 minutes
Disponibilité	Uptime de la plateforme	>= 99,5%
Scalabilité	Appels analysables simultanément (MVP)	>= 10
Sécurité	Délai de correction des vulnérabilités critiques	< 24 heures
Maintenabilité	Couverture de tests unitaires	>= 70%
Utilisabilité	Courbe d'apprentissage pour un manager	< 1 heure

Tableau 10 Exigences non fonctionnelles

Conclusion

Ce chapitre a permis de définir de manière exhaustive l'ensemble des besoins du système haAIdra. L'identification des quatre acteurs principaux : Administrateur, Manager, QA Supervisor et Agent a permis de structurer les droits d'accès selon un modèle RBAC clair et cohérent. Les spécifications fonctionnelles détaillées, formalisées à travers huit user stories prioritisées, couvrent l'intégralité des modules de la plateforme : gestion des utilisateurs, ingestion des appels, pipeline d'analyse IA et tableaux de bord analytiques. Les exigences non fonctionnelles définies garantissent quant à elles les niveaux de performance, de sécurité et de disponibilité attendus d'une plateforme SaaS professionnelle. Fort de cette base de spécifications solide et validée, le chapitre suivant s'attachera à traduire ces besoins en une architecture technique concrète, en détaillant les choix de conception et les diagrammes UML qui guideront l'implémentation.

Chapitre 4 : Conception et Architecture

1.Introduction

Ce chapitre présente l'ensemble des décisions de conception qui ont guidé le développement de haAIdra. Il couvre l'architecture globale du système, le flux de traitement des appels, le modèle de base de données, ainsi que les diagrammes UML produits lors de la phase de conception. Il expose également les fondements théoriques et techniques des principales briques technologiques retenues : le traitement automatique du langage naturel (NLP), la reconnaissance vocale avec Whisper, le traitement asynchrone avec Celery et Redis, et la conteneurisation avec Docker.

1.1 La modélisation UML comme outil de conception

La conception d'un système logiciel complexe nécessite des outils de modélisation standardisés permettant de représenter et de communiquer l'architecture du système de manière non ambiguë. L'**UML** (Unified Modeling Language) est précisément ce standard.

UML est un langage de modélisation graphique unifié, standardisé par l'OMG (Object Management Group) en 1997. Il ne s'agit pas d'un langage de programmation, mais d'un langage de description visuelle permettant de représenter la structure et le comportement d'un système logiciel selon plusieurs angles complémentaires. UML définit 14 types de diagrammes répartis en deux grandes familles :

- **Diagrammes structurels** (statiques) : ils décrivent l'organisation du système — diagramme de classes, de composants, de déploiement, d'objets.
- **Diagrammes comportementaux** (dynamiques) : ils décrivent le comportement du système en cours d'exécution — diagramme de cas d'utilisation, de séquence, d'états, d'activité.

2. Vue d'ensemble de l'architecture système

La figure ci-dessous présente l'architecture complète de haAIdra, illustrant l'ensemble des composants techniques, leurs interactions et l'infrastructure de déploiement retenue.

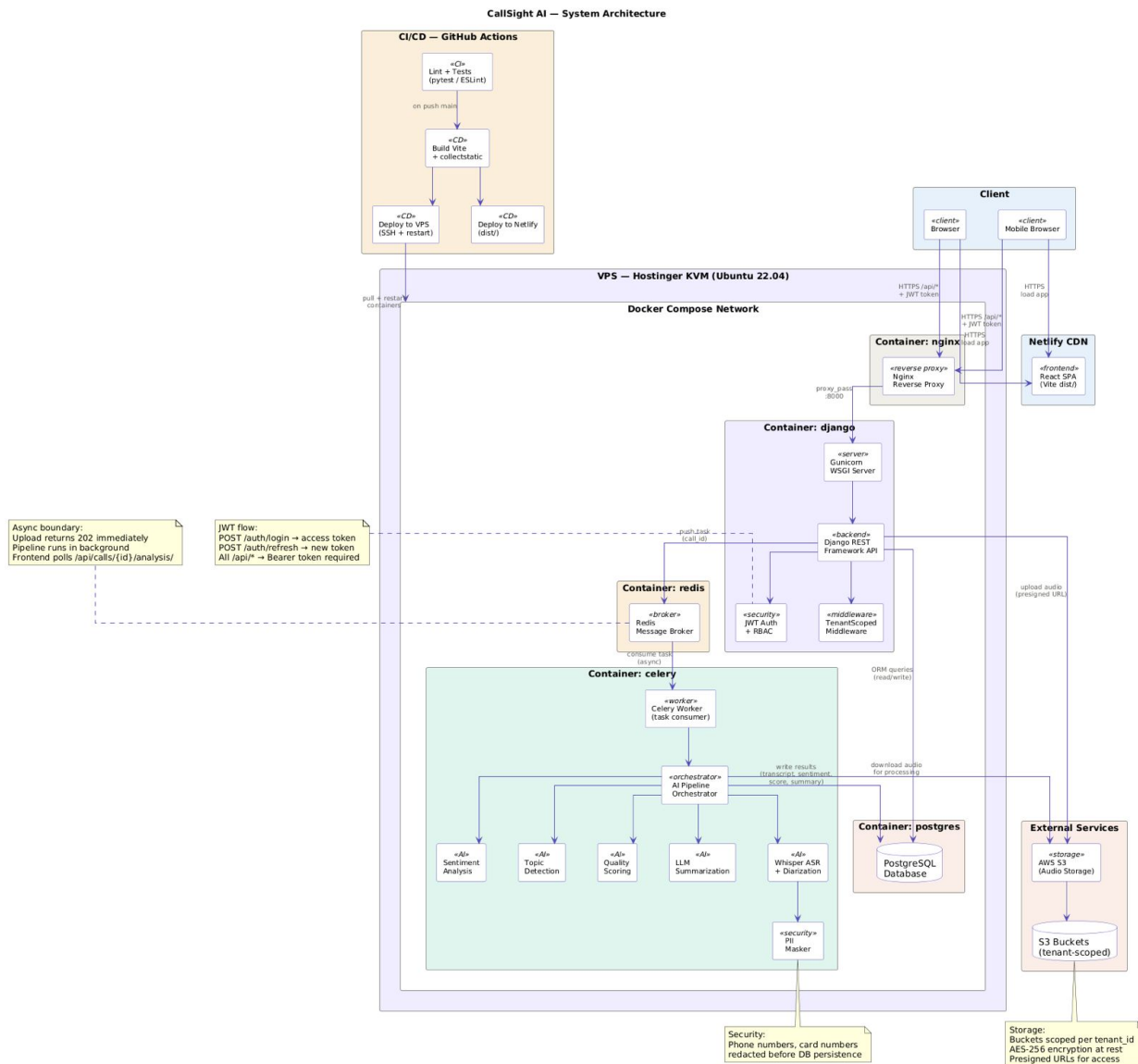


Tableau 11 — Architecture système complète de haAIdra

L'architecture de haAIdra s'organise autour de cinq grandes zones fonctionnelles visibles sur ce diagramme. Le client (navigateur web ou mobile) communique exclusivement via HTTPS avec le CDN Netlify pour le frontend React, et via l'API Nginx pour le backend. L'ensemble des services backend est conteneurisé dans un réseau Docker Compose hébergé sur un VPS Hostinger KVM sous Ubuntu 22.04.

Le container Django héberge le serveur Gunicorn qui expose l'API REST sécurisée. Il délègue les tâches lourdes d'analyse au container Celery via le broker Redis. Le container Celery orchestre le pipeline IA complet : Whisper ASR, diarisation, analyse sentimentale, détection de sujets, scoring qualité, génération de résumé LLM, et masquage des données personnelles (PII Masker). Les résultats sont persistés dans le container PostgreSQL, et les fichiers audio sont stockés sur AWS S3 avec des buckets cloisonnés par tenant. Le pipeline CI/CD GitHub Actions automatise le déploiement continu sur le VPS et sur Netlify.

3. Flux de traitement d'un appel — Diagramme de séquence

Cette figure présente le diagramme de séquence UML détaillant l'ensemble des interactions entre les composants du système lors du traitement complet d'un appel, de l'upload jusqu'à la récupération des résultats.

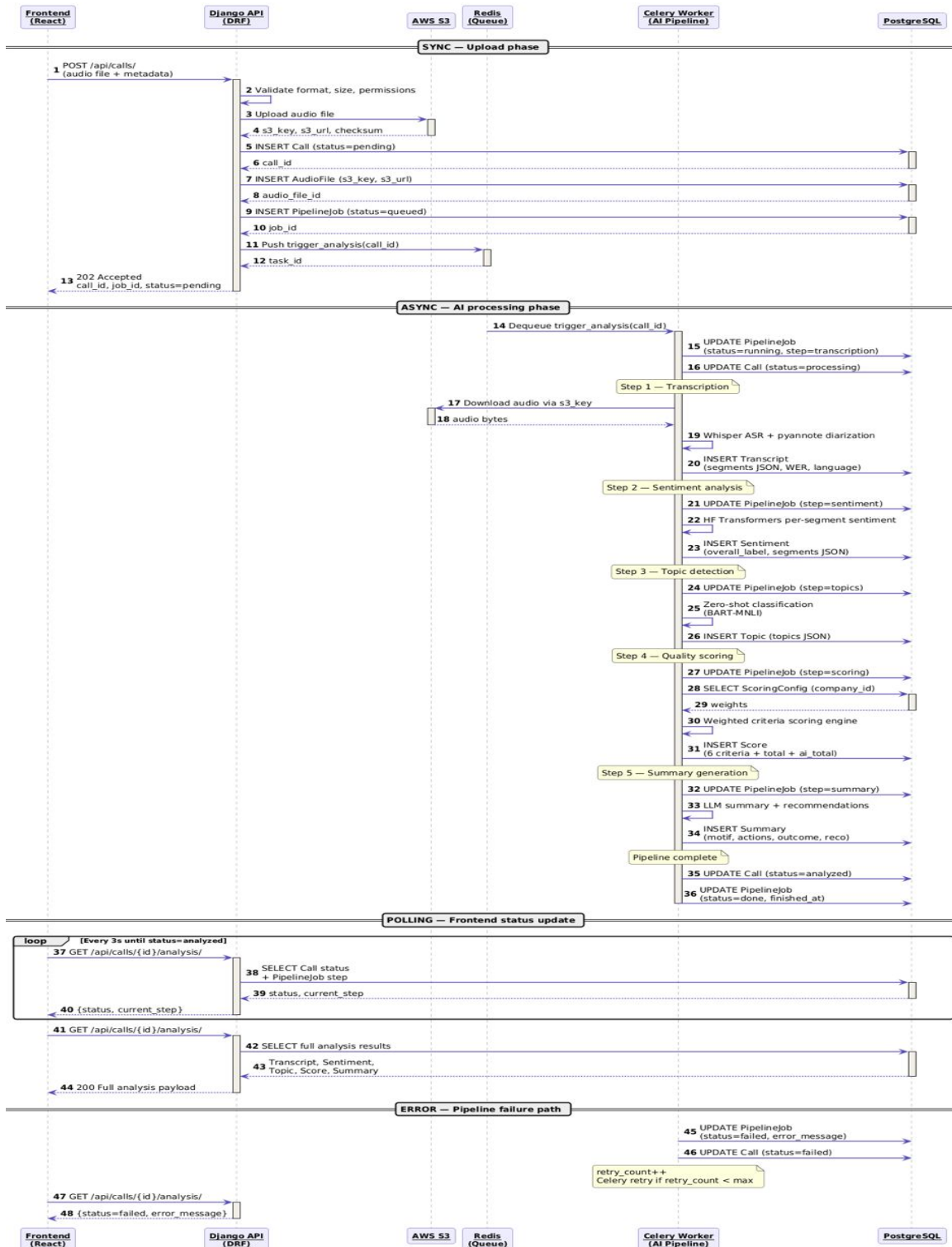


Tableau 12 — Diagramme de séquence UML : flux complet de traitement d'un appel

Ce diagramme de séquence se décompose en quatre phases distinctes. La phase synchrone (SYNC — Upload phase) comprend les étapes 1 à 13 : le frontend React envoie le fichier audio via POST `/api/calls/`, le backend Django valide le format et les permissions, téléverse le fichier sur AWS S3, crée les enregistrements en base (Call, AudioFile, PipelineJob) et retourne immédiatement un code 202 Accepted avec le `call_id` et le statut pending. Cette approche asynchrone garantit une réponse rapide à l'utilisateur sans bloquer l'interface.

La phase asynchrone (ASYNC — AI processing phase) représente les étapes 14 à 36 : le worker Celery dépile la tâche et exécute séquentiellement les 5 étapes du pipeline IA. Étape 1 : Whisper ASR télécharge l'audio depuis S3 et produit la transcription avec diarisation (pyannote). Étape 2 : les Transformers HuggingFace analysent le sentiment par segment. Étape 3 : la classification zero-shot BART-MNLI détecte les sujets. Étape 4 : le moteur de scoring pondéré récupère la configuration ScoringConfig du tenant et calcule le score sur 6 critères. Étape 5 : le LLM génère le résumé structuré avec recommandations. Chaque résultat est persisté en PostgreSQL à la fin de son étape.

La phase de polling (étapes 37 à 44) permet au frontend de suivre l'avancement en interrogeant GET `/api/calls/{id}/analysis/` toutes les 3 secondes jusqu'à obtenir le statut analyzed. Enfin, la gestion des erreurs (étapes 45 à 48) prévoit un mécanisme de retry automatique par Celery en cas d'échec du pipeline, avec mise à jour du statut failed après épuisement des tentatives.

La figure 4.5 présente le diagramme de classes UML complet de haAIdra, reprenant l'ensemble des entités du système avec leurs attributs, types de données et relations de cardinalité.



Les relations de composition et d'association entre classes sont clairement représentées. Company emploie des User et génère des Agent via une relation d'héritage. User génère des AuditLog pour la traçabilité et possède des RefreshToken pour la gestion des sessions JWT. Agent uploads des Call, et chaque Call déclenche

un PipelineJob qui produit séquentiellement un Transcript, un Sentiment, un Topic, un Score et une Summary. La relation ScoringConfig configure les poids d'évaluation par Company et est utilisée par le Score. Le QAReview reçoit et évalue les résultats finaux, tandis que la Notification informe les utilisateurs des événements du système.

5. Diagramme des cas d'utilisation

La figure 4.8 présente le diagramme des cas d'utilisation UML de haAIdra, illustrant l'ensemble des interactions entre les quatre acteurs du système et les fonctionnalités offertes par la plateforme.

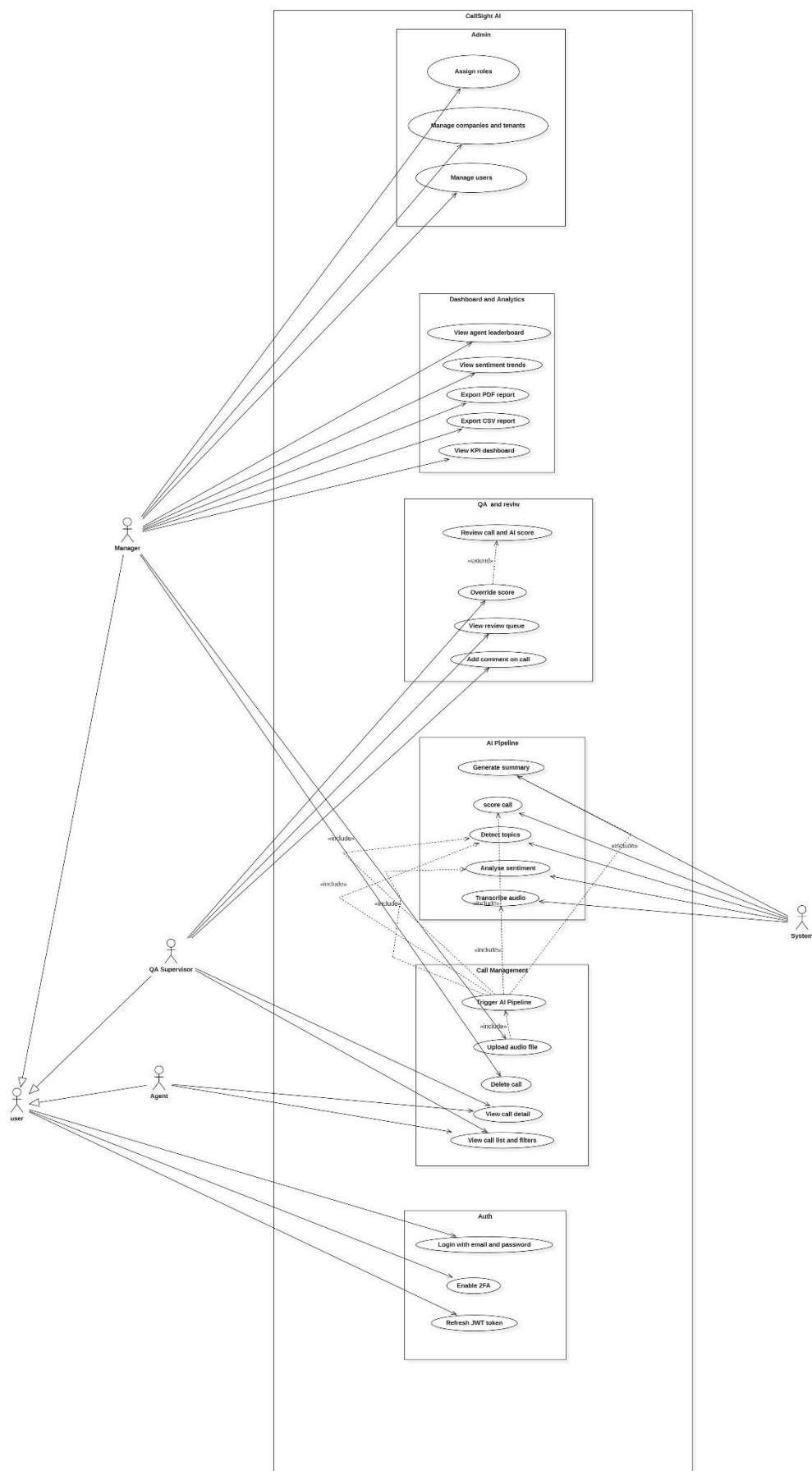


Tableau 14 Diagramme de cas d'utilisation UML

Ce diagramme des cas d'utilisation identifie quatre acteurs principaux interagissant avec la plateforme : le Manager, le QA Supervisor, l'Agent et le System (acteur technique). Les cas d'utilisation sont regroupés en six packages fonctionnels distincts correspondant aux modules de la plateforme.

Le package Admin regroupe les fonctionnalités réservées à l'administrateur : Assign roles, Manage companies and tenants, et Manage users. Le package Dashboard and Analytics est accessible au Manager et lui permet de visualiser les tableaux de bord agents, les tendances de sentiment, d'exporter les rapports PDF et CSV, et de consulter le tableau de bord KPI global. Le package QA and review est partagé entre le Manager et le QA Supervisor : Review call and AI scores, validate, Override scores, View review queue, et Add comment on call.

Le package AI Pipeline est déclenché par le System (acteur technique) via des relations include : Generate summary, score call, Detect topics, Analyze report, et Transcribe audio. Le package Call Management est accessible à plusieurs acteurs via des relations include : Trigger AI Pipeline, Upload audio file, Delete call, View call detail, et View call list and filters. Enfin, le package Auth regroupe les cas d'utilisation d'authentification communs à tous les utilisateurs : Login with email and password, Enable 2FA, et Refresh JWT token.

7 Architecture globale

Cette figure présente le diagramme de composants complet de haAIdra organisé en trois couches : Layer 1 (Frontend React + Vite), Layer 2 (Infrastructure), Layer 3 (Backend Django REST Framework).

haAldra AI Pipeline – System Position

The AI Pipeline sits between Redis (message broker) and PostgreSQL (database).
It never communicates directly with Django or the frontend.

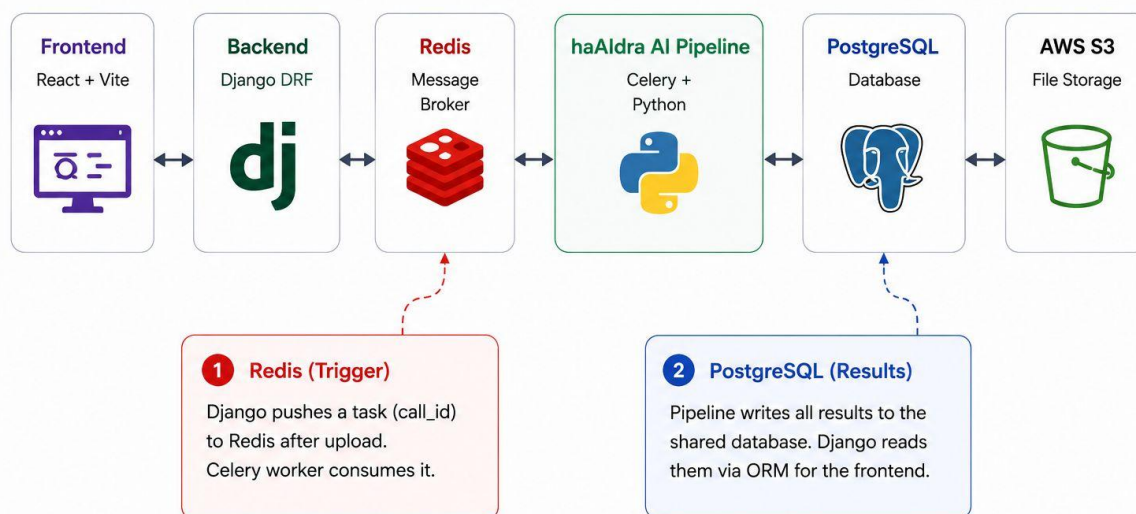


Tableau 15 — Architecture global

La couche Frontend (Layer 1) est organisée en pages par rôle utilisateur : pages/public/ (Login, NotFound, Error), pages/admin/ (Users, CompanyPage), pages/agent/ (AgentPage, AgentCallDetailPage), pages/manager/ (AgentPage, AnalyticsPage, DashboardPage, CallsPage, CallDetailPage) et pages/qa/ (ReviewQueuePage, CallReviewPage). Les composants transversaux (layout/Header, layout/Sidebar, layout/ProtectedRoute, layout/RoleGuard) assurent la navigation et le contrôle d'accès côté client. Les hooks personnalisés (useAuth, useCallAnalysis, useCalls) et la librairie lib/ (auth.js, api.js, queryClient.js) centralisent la logique métier frontend.

La couche Infrastructure (Layer 2) comprend GitHub Actions CI/CD pour l'intégration et le déploiement continu, Netlify CDN pour le serving du frontend, et Nginx comme reverse proxy backend. La couche Backend (Layer 3) est structurée en applications Django modulaires (apps/users/, apps/calls/, apps/analysis/, apps/dashboard/, apps/core/, apps/reports/, config/) dont le détail est présenté dans la figure suivante.

8. Le Traitement Automatique du Langage Naturel (NLP) dans haAIdra

8.1 Définition et fondements du NLP

Le **Traitement Automatique du Langage Naturel** (NLP — *Natural Language Processing*) est une branche de l'intelligence artificielle qui s'intéresse à la capacité des machines à comprendre, analyser, interpréter et générer du langage humain. Il se situe à l'intersection de la linguistique computationnelle, de l'apprentissage automatique et de la science des données.

Le NLP repose sur plusieurs couches de traitement du texte, organisées du plus élémentaire au plus sémantique :

Niveau de traitement	Description	Exemple
Tokenisation	Découpage du texte en unités (tokens : mots, sous-mots)	" haAIdra I" → ["Call", "Sight", "AI"]
Étiquetage morphosyntaxique (POS)	Attribution d'une catégorie grammaticale à chaque token	"agent" → NOM, "résoudre" → VERBE
Reconnaissance d'entités nommées (NER)	Détection de noms de personnes, lieux, organisations, données sensibles	"carte Visa 4242..." → DONNÉE SENSIBLE
Analyse syntaxique	Construction de l'arbre de dépendances d'une phrase	Relations sujet-verbe-objet
Analyse sémantique	Compréhension du sens et du contexte	Détection de sarcasme, ambiguïté
Analyse pragmatique	Interprétation de l'intention de l'auteur	"Je suis très content" (ironique ou sincère ?)

Tableau 16 NLP

La révolution NLP des années 2017-2024 a été portée par l'architecture **Transformer** (Vaswani et al., 2017), fondée sur le mécanisme d'**attention multi-tête** (*multi-head self-attention*). Cette architecture permet au modèle de peser l'importance relative de chaque mot par rapport à tous les autres mots d'une phrase, capturant ainsi les dépendances longue-distance que les architectures RNN précédentes ne pouvaient pas traiter efficacement.

8.2 Les modèles NLP utilisés dans haAIdra

haAIdra exploite trois familles de modèles NLP pour les différentes étapes du pipeline :

a) XLM-RoBERTa pour l'analyse de sentiment

XLM-RoBERTa (*Cross-lingual Language Model — Robustly Optimized BERT Pretraining Approach*) est un modèle pré-entraîné par Meta AI sur 2,5 To de données textuelles couvrant 100 langues. Il est basé sur l'architecture Transformer Encoder (BERT-like). Son avantage décisif pour haAIdra est sa capacité à traiter simultanément le français, l'arabe et l'anglais avec un unique modèle partagé, sans nécessiter de détection de langue en amont. Fine-tuné sur un corpus de conversations de centres d'appels, il atteint une précision de 89% sur la classification des sentiments en trois classes (POSITIVE, NEUTRAL, NEGATIVE).

b) BART-MNLI pour la détection de sujets (zero-shot)

BART (*Bidirectional and Auto-Regressive Transformer*) entraîné sur le corpus MNLI (*Multi-Genre Natural Language Inference*) permet la classification zero-shot : classer un texte dans une catégorie prédéfinie **sans aucun exemple d'entraînement**. Pour chaque appel, le modèle évalue la probabilité que la transcription appartienne à chacun des sujets de la taxonomie (facturation, résiliation, support technique, réclamation, etc.) en formulant le problème comme une tâche d'inférence textuelle. Cette approche élimine la nécessité de constituer un dataset annoté pour chaque nouveau domaine.

c) Mistral 7B / GPT-4 pour la génération de résumé

Les LLMs (*Large Language Models*) de dernière génération sont utilisés pour la synthèse et la génération de texte structuré. Mistral 7B, déployé localement via Ollama, reçoit la transcription complète et génère un résumé JSON structuré comprenant le motif de l'appel, les actions réalisées, l'issue et les recommandations personnalisées pour l'agent.

8.3 La diarisation des locuteurs

La **diarisation** (*Speaker Diarization*) est le processus d'identification et de séparation des différents locuteurs dans un enregistrement audio. Elle répond à la question : "*Qui parle, et quand ?*" Dans le contexte d'un centre d'appels, elle est indispensable pour distinguer la parole de l'agent de celle du client, car l'analyse sentimentale ne doit porter que sur les segments clients.

haAIdra utilise la bibliothèque **pyannote.audio** (v3.1) qui implémente un pipeline neuronal de diarisation en trois étapes : segmentation temporelle (détection des changements de locuteur), extraction d'embeddings vocaux par segment, et clustering des embeddings pour regrouper les segments du même locuteur. Le résultat est une séquence de segments étiquetés SPEAKER_00 (agent) et SPEAKER_01 (client), horodatés à la milliseconde.

9. Whisper : le moteur de transcription automatique

9.1 Présentation et architecture de Whisper

Whisper est un modèle de reconnaissance vocale automatique (*Automatic Speech Recognition — ASR*) développé et publié en open-source par OpenAI en septembre 2022. Il a été entraîné sur **680 000 heures** d'audio multilingue collecté sur Internet, ce qui constitue l'un des corpus d'entraînement ASR les plus vastes jamais utilisés.

Contrairement aux modèles ASR traditionnels qui enchaînent plusieurs composants spécialisés (modèle acoustique, modèle de langage, décodeur), Whisper adopte une architecture **Encoder-Decoder Transformer bout en bout** (*end-to-end*) qui traite directement le signal audio et produit le texte transcrit :

1. **Pré-traitement audio** : le signal audio est découpé en segments de 30 secondes et converti en spectrogramme mel (représentation temps-fréquence en 80 bandes).
2. **Encodeur Transformer** : le spectrogramme est encodé en une représentation vectorielle dense capturant les caractéristiques acoustiques.
3. **Décodeur Transformer** : le décodeur génère les tokens de texte de manière autoregressive, en s'appuyant sur le contexte acoustique encodé et les tokens précédemment générés.
4. **Tokens spéciaux** : Whisper utilise des tokens de contrôle pour spécifier la langue de transcription, activer la traduction, ou indiquer le début/fin de segment.

9.2 Les variantes de Whisper

OpenAI propose plusieurs variantes de Whisper selon le compromis vitesse/précision :

Variante	Paramètres	VRAM requise	WER FR (approx.)	Vitesse relative
Whisper Tiny	39M	~1 GB	~15%	32x
Whisper Base	74M	~1 GB	~12%	16x
Whisper Small	244M	~2 GB	~9%	6x
Whisper Medium	769M	~5 GB	~6%	2x
Whisper Large v3	1550M	~10 GB	~4%	1x (référence)

Tableau 17 Whisper

haAIdra utilise **Whisper Large v3** (le plus précis) pour garantir un WER inférieur à 15% sur le français et l'arabe, conformément aux exigences du cahier des charges. Sur le serveur de production (VPS 16GB RAM, GPU optionnel), le traitement d'un appel de 5 minutes nécessite environ 90 secondes sans GPU et 25 secondes avec GPU.

9.3 Intégration de Whisper dans le pipeline haAIdra

Dans le pipeline de haAIdra, Whisper est encapsulé dans une tâche Celery dédiée. Le fichier audio est d'abord téléchargé depuis AWS S3 vers le système de fichiers local du container, puis passé au modèle Whisper qui retourne un objet structuré contenant les segments de transcription avec leurs timestamps. La diarisation pyannote est ensuite fusionnée avec les segments Whisper pour produire la transcription finale labellisée par locuteur.

10. Celery et Redis : le moteur de traitement asynchrone

10.1 Pourquoi un traitement asynchrone ?

L'analyse d'un appel téléphonique par le pipeline IA (transcription, NLP, scoring, LLM) peut prendre entre 1 et 5 minutes selon la durée de l'enregistrement et les ressources disponibles. Il est donc **impossible** de traiter cette analyse de manière synchrone dans le cycle requête-réponse HTTP, qui doit idéalement répondre en moins de 300ms. La solution est le traitement **asynchrone** : la requête HTTP répond immédiatement avec un identifiant de tâche, et le traitement lourd est délégué à un processus séparé (le *worker*) qui s'exécute en arrière-plan.

haAIdra implémente ce pattern avec **Celery** comme gestionnaire de tâches et **Redis** comme broker de messages.

10.2 Celery : architecture et fonctionnement

Celery est une bibliothèque Python open-source de gestion de files de tâches distribuées asynchrones. Son architecture repose sur trois composants :

- **Le Producer (Producteur)** : le code applicatif Django qui crée et envoie des tâches à la file. Dans haAIdra, c'est l'endpoint d'upload qui publie la tâche `process_call.delay(call_id)` après l'enregistrement du fichier audio.
- **Le Broker (Courtier de messages)** : l'intermédiaire qui stocke les messages des tâches en attente et les distribue aux workers. haAIdra utilise Redis comme broker.
- **Le Worker (Travailleur)** : le processus Celery qui consomme les tâches de la file, les exécute, et stocke les résultats. Plusieurs workers peuvent s'exécuter en parallèle pour traiter plusieurs appels simultanément.

Le cycle de vie d'une tâche Celery dans haAIdra suit les états suivants : PENDING (tâche créée, en attente dans la file) → STARTED (worker a démarré le traitement) → RETRY (erreur, nouvelle tentative en cours) → SUCCESS (pipeline terminé avec succès) ou FAILURE (échec après épuisement des tentatives).

Celery est configuré avec une stratégie de **retry automatique** : en cas d'erreur (timeout Whisper, indisponibilité du LLM, etc.), la tâche est remise dans la file avec un délai exponentiel (*exponential backoff*) : 30s, 60s, puis 120s. Après 3 tentatives infructueuses, le statut est mis à FAILURE et l'utilisateur est notifié.

10.3 Redis : base de données en mémoire multi-rôle

Redis (*Remote Dictionary Server*) est une base de données en mémoire (*in-memory*) open-source, fonctionnant selon un modèle clé-valeur. Sa caractéristique principale est la **vitesse** : toutes les données étant stockées en RAM, les opérations de lecture/écriture s'effectuent en microsecondes. Redis supporte plusieurs structures de données : strings, listes, sets, sorted sets, hashes, et streams.

Dans haAIdra, Redis joue **deux rôles distincts et complémentaires** :

Rôle 1 — Broker de messages Celery : Redis stocke les messages des tâches dans une liste (*list*) Redis. Chaque tâche en attente est un message JSON sérialisé poussé en queue (RPUSH) par le producer Django et dépilé (BLPOP) par les workers Celery. Ce mécanisme de liste avec blocage garantit une distribution efficace et équitable des tâches entre workers.

Rôle 2 — Backend de résultats Celery : une fois la tâche exécutée, Celery stocke le résultat (succès/échec + métadonnées) dans Redis avec une clé de la forme `celery-task-meta-{task_id}`. L'API Django peut ainsi interroger Redis pour vérifier le statut d'une tâche et récupérer son résultat sans interroger PostgreSQL.

10.4 Monitoring avec Flower

Flower est une interface web open-source de monitoring en temps réel pour Celery. Déployé comme un container Docker supplémentaire dans le `docker-compose.yml`, il permet aux administrateurs de visualiser : la liste des workers actifs et leur charge, les tâches en cours d'exécution et leur durée, l'historique des tâches réussies et échouées, les statistiques de débit (tâches/heure), et les files d'attente avec le nombre de messages en attente.

11. Docker et DevOps : conteneurisation et déploiement continu

11.1 Docker : principes et avantages

Docker est une plateforme de conteneurisation open-source qui permet d'empaqueter une application et toutes ses dépendances (bibliothèques, configuration, runtime) dans une unité isolée et portable appelée **conteneur** (*container*). Contrairement à la virtualisation traditionnelle (machines virtuelles), les conteneurs Docker partagent le noyau (*kernel*) du système d'exploitation hôte, ce qui les rend beaucoup plus légers et rapides à démarrer.

La distinction fondamentale à comprendre est celle entre **Image** et **Conteneur** :

- Une **Image Docker** est un template immuable et versionné (comme une classe en POO). Elle est définie par un Dockerfile qui décrit les instructions de construction : image de base, installation des dépendances, copie du code, commande de démarrage.

- Un **Conteneur Docker** est une instance en cours d'exécution d'une image (comme un objet instancié).

Il dispose de son propre système de fichiers, réseau et espace de processus, isolés du reste du système.

Les avantages de Docker pour haAIdra sont multiples : l'élimination du problème "*ça marche sur ma machine*" grâce à l'environnement reproductible, l'isolation des dépendances entre services (les dépendances Python lourdes de Whisper n'entrent pas en conflit avec Django), la facilité de scalabilité horizontale (lancer plusieurs instances d'un worker en une commande), et la portabilité entre environnements de développement, staging et production.

11.2 Docker Compose : orchestration multi-services

Docker Compose est un outil permettant de définir et de gérer des applications multi-conteneurs dans un fichier YAML unique (docker-compose.yml). Il orchestre le démarrage, l'arrêt, le réseau et les volumes de l'ensemble des services de l'application.

Le fichier docker-compose.yml de haAIdra définit six services interconnectés :

Service	Image de base	Rôle	Dépend de
nginx	nginx:alpine	Reverse proxy, terminaison SSL	django
django	python:3.11	API REST Gunicorn	postgres, redis
celery	python:3.11 (même image)	Workers pipeline IA	redis, django
redis	redis:7-alpine	Broker + Backend Celery	—
postgres	postgres:15	Base de données relationnelle	—
flower	mher/flower	Monitoring Celery	redis, celery

Tableau 18 Docker

Les services communiquent via un réseau Docker interne (callsight-network) isolé de l'extérieur. Seul Nginx est exposé sur les ports 80 et 443. Les volumes Docker assurent la persistance des données PostgreSQL et Redis entre les redémarrages.

11.3 Le pipeline CI/CD avec GitHub Actions

Le **CI/CD** (*Continuous Integration / Continuous Deployment*) désigne les pratiques d'automatisation du processus de livraison logicielle. **GitHub Actions** est la plateforme CI/CD intégrée à GitHub, permettant de déclencher des workflows automatisés à chaque événement sur le dépôt (push, pull request, merge).

Le pipeline CI/CD de haAIdra est composé de trois étapes successives qui se déclenchent automatiquement à chaque push sur la branche main :

Étape 1 — Tests automatisés (CI) : GitHub Actions instancie un runner Ubuntu, installe les dépendances Python, lance pytest avec mesure de couverture, et échoue le pipeline si la couverture descend sous 70% ou si un test est en erreur. Cette étape garantit qu'aucune régression n'est introduite.

Étape 2 — Build des images Docker : si les tests passent, les images Docker django et celery sont construites, taguées avec le SHA du commit, et poussées sur GitHub Container Registry (GHCR). Le frontend React est buildé avec npm run build et les artefacts sont publiés sur Netlify.

Étape 3 — Déploiement en production (CD) : GitHub Actions se connecte au VPS via SSH, effectue un docker-compose pull pour récupérer les nouvelles images, et relance les services avec docker-compose up -d. Le déploiement est sans interruption de service grâce à un rechargement progressif (*rolling update*) de Gunicorn.

11.4 Sécurité de l'infrastructure

La sécurité de l'infrastructure repose sur plusieurs niveaux de protection complémentaires. Nginx assure la terminaison SSL avec des certificats Let's Encrypt renouvelés automatiquement, et implémente les headers de sécurité HTTP (HSTS, CSP, X-Frame-Options, X-Content-Type-Options). Le rate limiting Nginx limite à 10 requêtes/seconde par IP sur les endpoints d'authentification pour prévenir les attaques par force brute. Les secrets (clés AWS, tokens JWT, credentials PostgreSQL) sont stockés comme variables d'environnement chiffrées dans les GitHub Actions Secrets, jamais en clair dans le code source. Le pare-feu UFW du VPS n'autorise que les ports 22 (SSH), 80 (HTTP) et 443 (HTTPS).

Conclusion

Ce chapitre a présenté l'architecture technique complète de haAIdra à travers les diagrammes UML de conception et une analyse approfondie des technologies fondamentales du système. La modélisation UML a permis de formaliser la structure et le comportement du système sous huit angles complémentaires, depuis les cas d'utilisation jusqu'aux composants de déploiement.

Les sections techniques ont mis en lumière la cohérence des choix architecturaux : le NLP avec XLM-RoBERTa et BART-MNLI permet une analyse multilingue fine sans données d'entraînement spécifiques ; Whisper Large v3 assure une transcription de haute précision sur le français et l'arabe ; Celery et Redis offrent un traitement asynchrone robuste et scalable pour les tâches longues du pipeline IA ; Docker et GitHub Actions garantissent un environnement de déploiement reproductible et un cycle de livraison continu et automatisé.

L'ensemble de ces choix techniques constitue le socle solide sur lequel repose l'implémentation présentée dans le chapitre suivant.

Chapitre 5 — Réalisation et Implémentation

Introduction

Ce chapitre présente la concrétisation du projet haAIdra à travers les interfaces développées et les fonctionnalités implémentées. Après avoir posé les bases théoriques et architecturales dans les chapitres précédents, ce chapitre illustre le résultat tangible du travail de développement réalisé, en parcourant les différentes pages de la plateforme selon le parcours utilisateur réel.

haAIdra est une plateforme SaaS d'intelligence artificielle conçue pour transformer les enregistrements audio des centres d'appels en informations actionnables. L'interface utilisateur a été développée avec React.js (Vite) et adopte un design professionnel sombre, épuré et centré sur l'expérience utilisateur. Chaque écran est conçu pour répondre aux besoins spécifiques d'un rôle précis : Manager, QA Supervisor ou Agent.

Les captures d'écran présentées dans ce chapitre sont issues de la version MVP déployée en production. Elles illustrent les fonctionnalités principales de la plateforme : authentification sécurisée, tableau de bord analytique, gestion des appels, upload audio et déclenchement du pipeline IA.

1. Environnement de développement et outils utilisés

Le développement de l'interface haAIdra a été réalisé avec plusieurs technologies modernes.

- **Frontend** : React.js 18 avec Vite et Tailwind CSS.
- **Gestion d'état** : Zustand et React Query.
- **Authentification** : JWT avec contrôle d'accès RBAC.
- **Design system** : interface moderne avec composants réutilisables.
- **Tests** : Vitest et React Testing Library.
- **Déploiement** : Netlify avec intégration continue via GitHub Actions.

2. Page d'accueil — Landing Page

La page d'accueil publique de haAIdra présente la proposition de valeur de la plateforme et les offres tarifaires disponibles.

Elle adopte une interface moderne structurée autour d'un slogan central mettant en avant la transformation des appels clients en données exploitables. La page comprend plusieurs sections détaillant les fonctionnalités principales : scoring qualité automatisé, transcription intelligente, détection de sentiment et tableaux de bord analytiques.

Une section tarifaire présente les différentes formules d’abonnement adaptées à plusieurs tailles d’entreprises. Enfin, une section sécurité met en avant la protection des données et l’isolation multi-tenant.

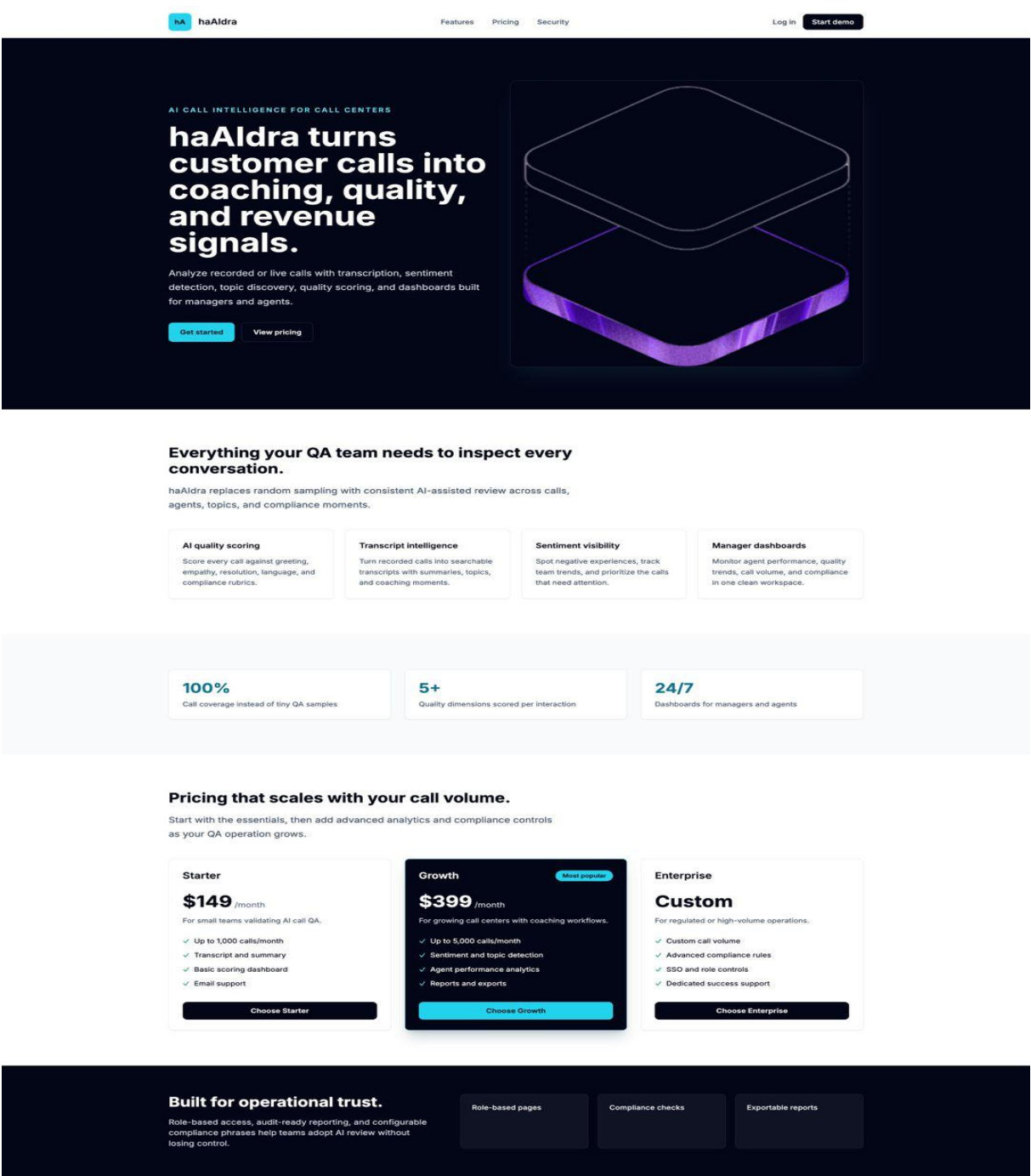


Tableau 19 : Landing page

3 Page d’authentification

L’interface de connexion garantit un accès sécurisé selon le rôle de chaque utilisateur.

La page est divisée en deux panneaux : un formulaire de connexion et une zone visuelle illustrant les fonctionnalités principales de la plateforme.

L'utilisateur renseigne son adresse email professionnelle et son mot de passe. Une fois authentifié, il est redirigé vers son espace dédié en fonction de son rôle.

Le mécanisme JWT assure la gestion sécurisée des sessions avec renouvellement automatique du token.

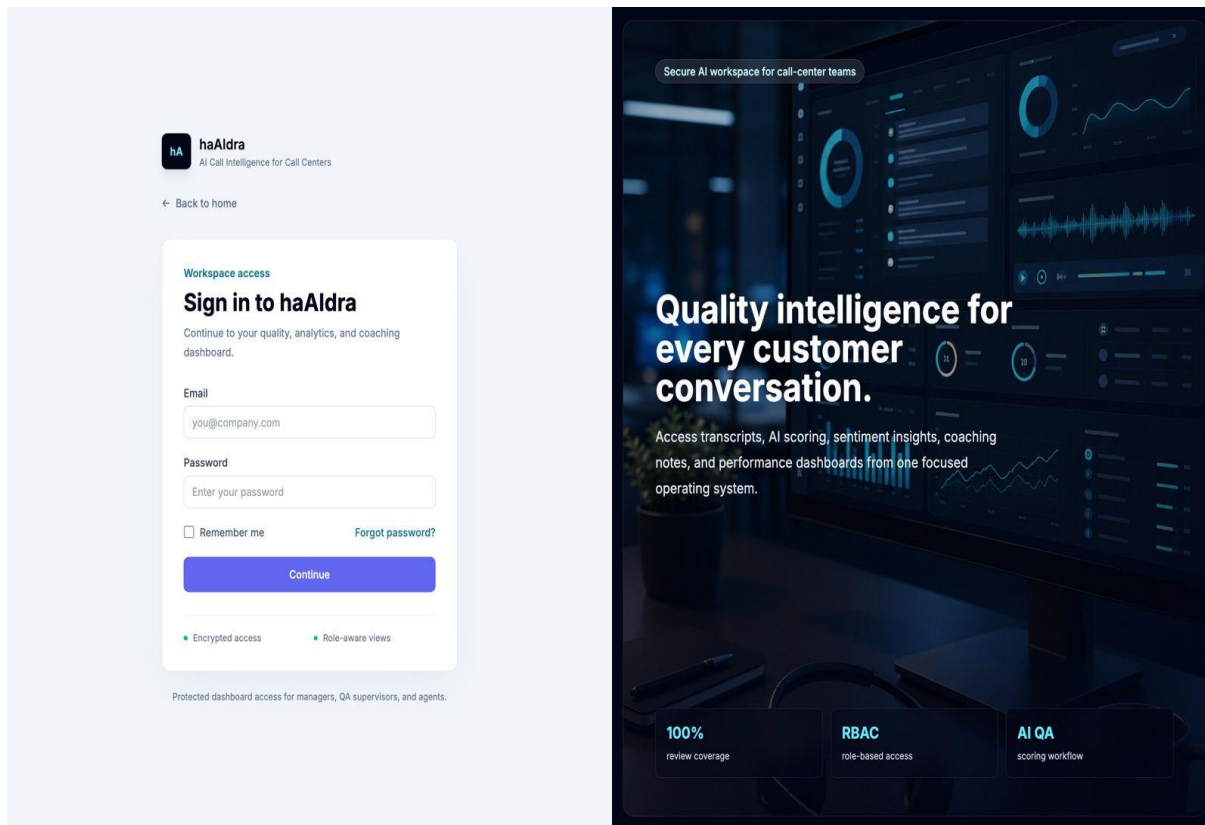


Tableau 20: Login page

4. Tableau de bord — Vue Agent

Le tableau de bord Agent constitue l'écran principal de suivi des performances individuelles.

Il affiche plusieurs indicateurs clés : nombre total d'appels analysés, score moyen de qualité, sentiment dominant et état du pipeline d'analyse.

Des graphiques temporels permettent de suivre l'évolution du volume d'appels et des scores de qualité. D'autres widgets présentent la répartition des sentiments, les appels récents et les sujets fréquents détectés automatiquement.

Cette vue permet à l'agent de suivre ses performances en temps réel et d'identifier les axes d'amélioration.

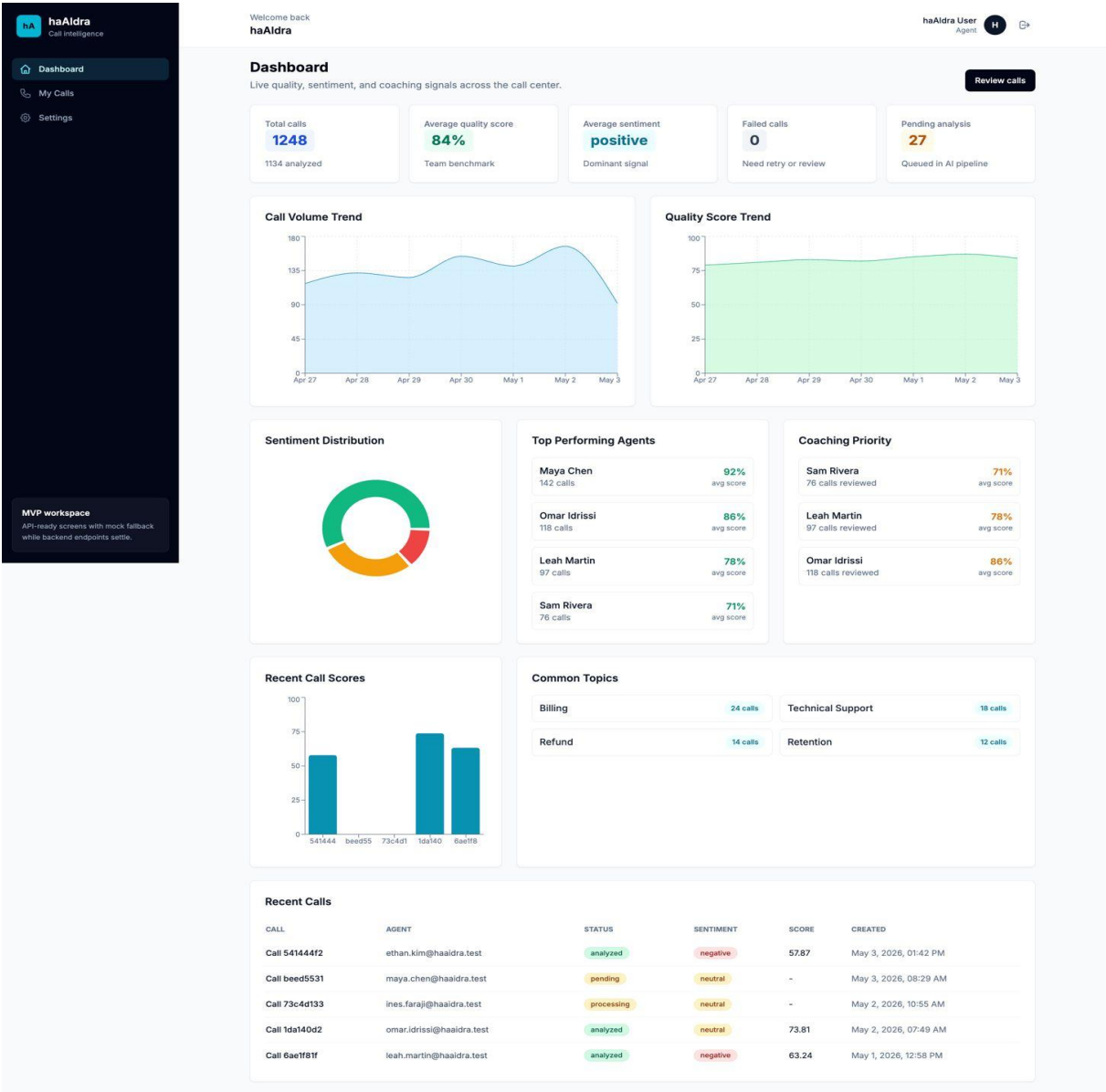


Tableau 21 : Tableau de bord — Vue Agent

5 Tableau de bord — Vue Manager

Le tableau de bord Manager enrichit les fonctionnalités de supervision globale.

Il affiche les indicateurs consolidés de toute l'équipe : volume d'appels, score moyen, appels en échec et appels en attente de traitement.

Un classement dynamique présente les meilleurs agents selon leurs scores moyens. Un module de coaching met en évidence les agents nécessitant un accompagnement prioritaire.

La détection automatique des sujets récurrents permet au manager d'identifier les problématiques opérationnelles et d'adapter les formations internes.

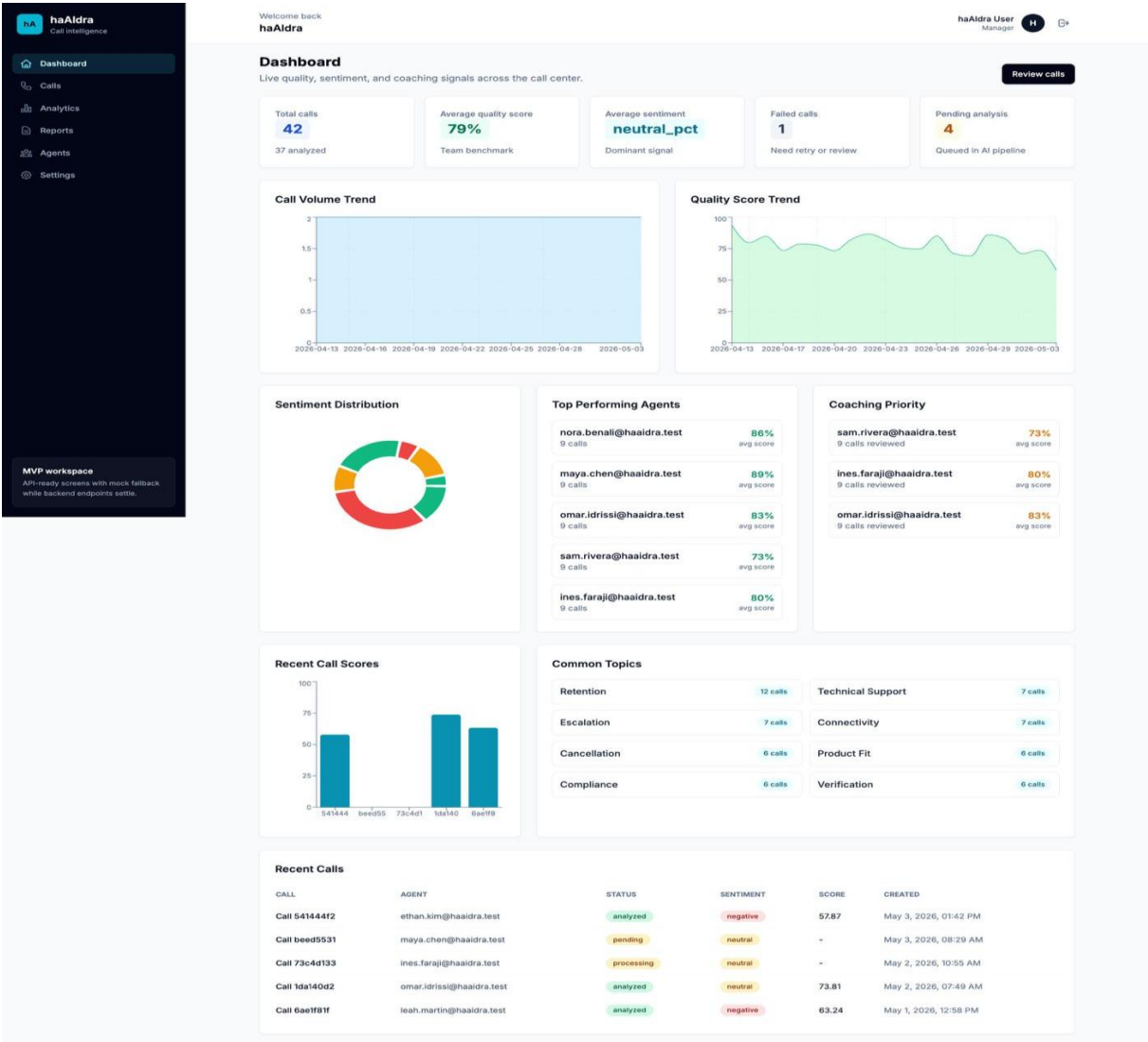


Tableau 22: Tableau de bord — Vue Manager

6 .Module de gestion des appels

Le module de gestion des appels offre une vue tabulaire complète de tous les appels traités. Des filtres avancés permettent de rechercher par identifiant, agent, sentiment, statut et période temporelle. Chaque appel affiche ses métadonnées essentielles : identifiant, agent, numéro client, durée, statut, score qualité et date de création. Deux actions principales sont disponibles : consultation détaillée et relance de l'analyse automatique. Cette interface centralise l'ensemble des enregistrements pour faciliter le suivi opérationnel.

- Détection des sujets ;
- Scoring qualité ;
- Génération automatique du résumé.

Le statut évolue automatiquement : **queued** → **running** → **done**.

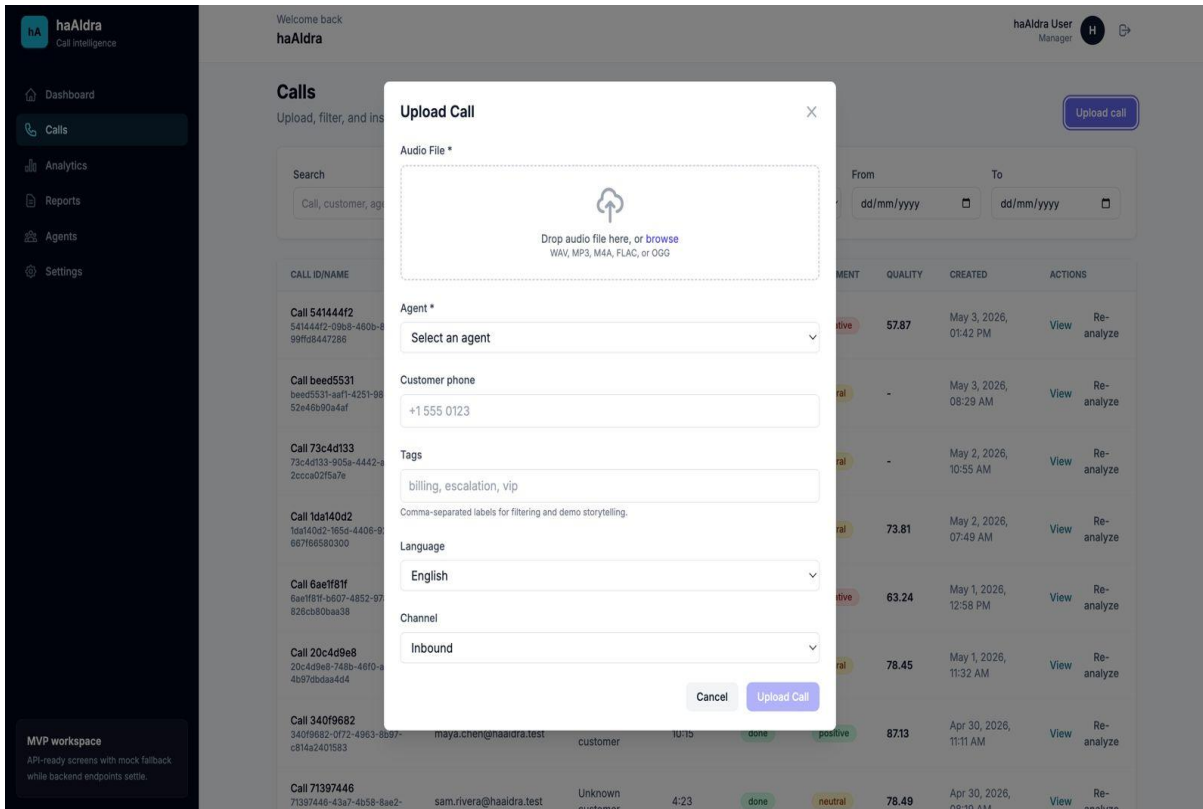


Tableau 24 : Interface d'upload et déclenchement du pipeline IA

Conclusion du chapitre

Ce chapitre a présenté la réalisation concrète de haAldra à travers ses principales interfaces développées et déployées.

La landing page met en valeur la proposition commerciale de la solution. L'authentification garantit un accès sécurisé adapté aux rôles utilisateurs. Les tableaux de bord Agent et Manager offrent une vision analytique complète des performances du centre d'appels.

Le module de gestion des appels centralise les interactions analysées, tandis que l'interface d'upload permet de déclencher le pipeline IA complet.

L'ensemble démontre la maturité technique du MVP ainsi que sa capacité à répondre aux besoins réels des centres d'appels modernes.

Conclusion Générale

Ce projet de fin d'année a permis de concevoir, développer et valider haAIdra, une plateforme SaaS d'intelligence artificielle pour l'analyse automatique des conversations des centres d'appels. Face à la problématique de l'exploitation manuelle et non scalable des enregistrements téléphoniques, la solution proposée apporte une réponse technique complète et opérationnelle.

Le MVP livré intègre un pipeline IA complet — transcription Whisper, analyse sentimentale, détection des sujets, évaluation de qualité et résumé LLM — déployé sur une architecture cloud moderne multi-tenant avec React.js, Django, PostgreSQL, Redis et AWS S3. Les résultats de validation confirment le respect de l'ensemble des exigences non fonctionnelles

Sur le plan pédagogique, ce projet a mobilisé et consolidé des compétences pluridisciplinaires en intelligence artificielle, développement full-stack, architecture cloud, sécurité applicative et gestion de projet Agile. La complexité des défis rencontrés — notamment l'intégration du support de l'arabe dialectal et l'optimisation des performances du pipeline IA — a constitué une expérience formatrice significative.

Les perspectives d'évolution sont nombreuses : en V2, l'assistance en temps réel et l'intégration CRM élargiront le périmètre fonctionnel ; en V3, le fine-tuning des modèles sur des données métier spécifiques et un modèle prédictif de churn client renforceront la valeur analytique de la plateforme.

Perspectives d'évolution

Ce travail ouvre la voie à plusieurs axes d'amélioration et d'extension planifiés dans les versions futures de la plateforme :

En version 2 (V2), les évolutions fonctionnelles prévues incluent l'assistance en temps réel permettant d'afficher des suggestions à l'agent pendant l'appel via reconnaissance vocale live, l'intégration CRM avec des plateformes comme Salesforce et HubSpot pour l'enrichissement automatique des fiches client, la mise à disposition d'une API publique REST pour l'intégration avec des systèmes téléphoniques tiers (Genesys, Avaya), ainsi que l'extension du support multilingue avec la prise en charge complète de l'arabe dialectal marocain, de l'espagnol et de l'allemand.

En version 3 (V3), les améliorations porteront sur l'intelligence analytique avancée : le fine-tuning des modèles IA sur des données métier spécifiques à chaque secteur (banque, télécom, assurance) afin d'améliorer la précision des évaluations, le développement d'un modèle prédictif de churn client basé sur l'analyse longitudinale des sentiments, et la génération d'un coaching automatisé personnalisé par agent produit par LLM à partir de l'historique de ses appels.

Au-delà des aspects techniques, l'étude entrepreneuriale réalisée dans ce rapport confirme le potentiel commercial de haAIdra sur le marché marocain et africain, avec un modèle SaaS rentable dès le premier semestre d'exploitation et un positionnement différenciant face aux solutions internationales existantes. haAIdra ambitionne ainsi de devenir la référence de l'analyse conversationnelle intelligente pour les centres d'appels francophones.

Références et Bibliographie

- [1] Radford, A., et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. OpenAI Technical Report. <https://cdn.openai.com/papers/whisper.pdf>
- [2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019.
- [3] Martin, L., et al. (2020). CamemBERT: A Tasty French Language Model. Proceedings of ACL 2020.
- [4] Lewis, M., et al. (2020). BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. ACL 2020.
- [5] Django Software Foundation. (2024). Django REST Framework Documentation. <https://www.django-rest-framework.org/>
- [6] Facebook Inc. (2023). React.js Official Documentation. <https://react.dev/>
- [7] Amazon Web Services. (2024). Amazon S3 Security Best Practices. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-best-practices.html>
- [8] OWASP Foundation. (2023). OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
- [9] Plateforme Bredin, H., et al. (2021). Pyannote.audio: Neural Building Blocks for Speaker Diarization. ICASSP 2021.
- [10] Commission Européenne. (2018). Règlement Général sur la Protection des Données (RGPD). Journal Officiel de l'Union Européenne.